Towards High-Goodput LLM Serving with Prefill-decode Multiplexing

Yukang Chen*
chenyukang@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Weihao Cui*
weihao@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China
National University of Singapore
Singapore

Han Zhao*
zhao-han@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Ziyi Xu
xzy2022@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Xiaoze Fan
jasonfxz@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Xusheng Chen
michael.xschen@gmail.com
Researcher
Shanghai, China

Yangjie Zhou
yj_zhou@nus.edu.sg
National University of Singapore
Singapore

Shixuan Sun
sunshixuan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Bingsheng He
dcsheb@nus.edu.sg
National University of Singapore
Singapore

Quan Chen†
chen-quan@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Abstract

Large Language Model (LLM) serving must meet stringent Service Level Objectives (SLOs) for both the prefill and decode phases. Some existing solutions disaggregate the two phases, causing potential resource idleness or compute redundancy. Others split the prefill phase into chunks and fuse it with decode iteration, creating a dilemma between SLO compliance and high utilization. To address these issues, an efficient serving system should dynamically adapt compute allocation, decouple compute from memory management, and execute prefill and decode independently. We present MuxWise, an LLM serving framework that adopts a new paradigm, intra-GPU prefill-decode multiplexing, to meet these requirements. To fully exploit the paradigm, MuxWise integrates a bubble-less multiplex engine, a contention-tolerant estimator, and an SLO-aware dispatcher. Evaluation shows that MuxWise improves peak throughput under SLO guarantees by an average of 2.20× (up to 3.06×) over state-of-the-art baselines.

CCS Concepts: **• Computer systems organization → Single instruction, multiple data**; **Cloud computing**; **• Software and its engineering → Process management**.

Keywords: LLM Serving, PD-Multiplexing, Goodput

ACM Reference Format:
Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaoze Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen. 2026. Towards High-Goodput LLM Serving with Prefill-decode Multiplexing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA*. ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3779212.3790236

*Equal contribution.
†Corresponding author.

ASPLOS '26, Pittsburgh, PA, USA

ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790236

1 Introduction

Large language models (LLM) services now perform well across diverse workloads [19, 30, 33]. At the request level, an LLM processes input in two phases: a prefill phase that produces the first token, followed by a decode phase that iteratively generates the remaining tokens. The ratio of input length (prefill) to output length (decode) varies across tasks [6, 43]. At the application level, tasks such as chatbot services or agent-based workloads [34] often consist of multiple turns of requests with shared context.

通过预填充解码复用实现高吞吐量 LLM 服务

陈宇康* chenyukang@sjtu.edu.cn 上海交通大学 中国上海

崔伟豪* weihao@sjtu.edu.cn 上海交通大学 中国上海 新加坡国立大学 新加坡

韩昭* zhao-han@cs.sjtu.edu.cn 上海交通大学 中国上海

徐子怡 xzy2022@sjtu.edu.cn 上海交通大学 中国上海

范晓泽 jasonfxz@sjtu.edu.cn 上海交通大学 中国上海

陈旭升 michael.xschen@gmail.com 研究员 中国上海

周洋杰 yj_zhou@nus.edu.sg 新加坡国立大学 新加坡

孙世轩 sunshixuan@sjtu.edu.cn 上海交通大学 中国上海

何秉生 dcsheb@nus.edu.sg 新加坡国立大学 新加坡

陈泉† chen-quan@cs.sjtu.edu.cn 上海交通大学 中国上海

抽象的

大型语言模型 (LLM) 服务必须满足预填充和解码阶段严格的服务级别目标 (SLO)。一些现有的解决方案将这两个阶段分开，导致潜在的资源闲置或计算冗余。其他人将预填充阶段拆分为多个块，并将其与解码迭代融合，从而在 SLO 合规性和高利用率之间造成了困境。为了解决这些问题，高效的服务系统应该动态调整计算分配，将计算与内存管理分离，并独立执行预填充和解码。我们提出了 MuxWise，一种 LLM 服务框架，它采用新的范式，即 GPU 内预填充解码复用，来满足这些要求。为了充分利用该范式，MuxWise 集成了无气泡多路复用引擎、耐争用估计器和 SLO 感知调度程序。评测显示 MuxWise 在 SLO 保证下将峰值吞吐量比最先进的基准平均提高了 2.20×(至 3.06×)。

CCS 概念： • 计算机系统组织→单指令，多数据；云计算； • 软件及其工程 → 流程管理。

关键词：LLM 服务、PD 复用、Goodput

ACM参考格式：
陈宇康、崔伟豪、赵涵、徐子怡、范晓泽、陈旭升、周杨杰、孙世轩、何冰生和陈泉。 2026.通过预填充解码多路复用实现高产出 LLM 服务。第 31 届 ACM 国际编程语言和操作系统架构支持会议论文集，第 2 卷 (ASPLOS’26)，2026 年 3 月 22-26 日，美国宾夕法尼亚州匹兹堡。 ACM，美国纽约州纽约市，18 页。 https://doi.org/10.1145/3779212.3790236

1 简介

大型语言模型 (LLM) 服务现在在不同的工作负载中表现良好 [19,30,33]。在请求级别，LLM 分两个阶段处理输入：生成第一个令牌的预填充阶段，然后是迭代生成剩余令牌的解码阶段。输入长度（预填充）与输出长度（解码）的比率因任务而异[6, 43]。在应用程序级别，诸如聊天机器人服务或基于代理的工作负载[34]之类的任务通常由具有共享上下文的多轮请求组成。

*Equal contribution.
†Corresponding author.

ASPLOS '26, Pittsburgh, PA, USA

ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790236

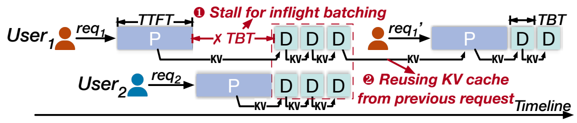


Figure 1. A typical workflow in LLM serving systems: `User1` sends two consecutive requests, with the second reusing the context from the first. `User2` sends 1 request.

To achieve high throughput for serving these workloads, existing LLM serving systems employ several optimizations. Figure 1 presents a typical workflow. While requests arrive at different times, inflight batching stalls the ongoing decode phase to prefill new requests and then processes all decode iterations together in a single batch. It greatly improves compute utilization for the memory-intensive decode phase [47]. Since multi-turn requests share context, LLM serving systems reuse intermediate results (i.e., the KV cache) both within and across requests through a KV cache pool [26, 52].

LLM services also impose stringent Service Level Objectives (SLOs). For instance, chatbot typically requires Time-To-First-Token (TTFT) under 500 ms for prefill and Time-Between-Tokens (TBT) under 100 ms for decode. Since prefill and decode interleave in an LLM serving system, SLO violations may arise. In Figure 1, inflight batching stalls ongoing decode. A long prefill can thus delay decode, potentially violating its SLO. To sustain high goodput–peak throughput with SLO guarantees–existing methods fall into two categories: disaggregated serving [32, 53] and chunked prefill [1].

As for disaggregated serving, Splitwise [32] separates the prefill and decode phases into distinct instances for SLO guarantees, which has two drawbacks. Firstly, it cannot adapt to serving dynamics. In Splitwise, GPUs are statically allocated at initialization. Under fluctuating request loads and diverse serving patterns, it often leads to resource underutilization. Secondly, it decreases goodput due to shrinking the KV cache pool. With the same number of GPUs, disaggregation allocates a separate cache pool for each instance, reducing the effective cache pool size. This lowers cache hit rate [45] (e.g., from 36.6% to 4.2%), leading to unnecessary recomputation and degraded goodput. Furthermore, while LoongServe [44] supports dynamic GPU allocation based on the request sequence length and execution phase, it cannot support the cross-request KV cache reuse, incurring significant recomputation overhead in multi-turn workloads.

Chunked-prefill [1] is another approach to meet decode SLOs. It splits the prefill phase into chunks within each GPU and fuses each chunk with a decode iteration. To ensure computational equivalence, each chunk reads the KV cache generated by all previous chunks. It ensures the decode SLOs by capping the token budget, defined as the sum of new tokens from the prefill chunk and the decode batch. By tuning the chunk and decode batch sizes, it adapts to serving dynamics. Since it avoids disaggregation, it also prevents goodput loss from a reduced KV cache pool.

Unfortunately, chunking is not a free lunch. It creates a dilemma between SLO compliance and high utilization. Because prefill chunk and decode iteration must execute together, the token budget governs both decode SLO attainment and GPU saturation. Yet, finding a sweet budget in practice is infeasible. E.g., deploying a 70B LLM on 8 A100 GPUs requires a 4K budget to saturate the GPU, which is 8× larger than the SLO-compliant budget (256 for a 100 ms TBT SLO). Moreover, TBT in chunk-prefill is inflated by repetitive KV cache access from the prefill chunk. With extremely long reused context, common in multi-turn workloads, chunked-prefill may even fail to meet SLO guarantees (§2.3.2). Ultimately, chunked-prefill cannot sustain high goodput.

Achieving high-goodput LLM serving requires more flexible compute management. We propose intra-GPU prefill-decode (PD) multiplexing as a promising new serving paradigm. Specifically, the prefill and decode phases are executed on different streaming multiprocessors (SMs) within the GPUs. In the new paradigm, 1) compute partitions can be reconfigured with low overhead to adapt to serving dynamics; 2) multiplexed phases share GPU memory, keeping the KV cache pool efficient; 3) with spatial sharing, prefill and decode execute independently, avoiding the tradeoff between SLO compliance and utilization.

Realizing this paradigm is non-trivial. Firstly, phase coordination is still required to enable inflight batching and improve compute utilization. However, since prefill and decode latencies differ significantly, naive coordination often leaves GPU bubbles. Secondly, spatial multiplexing introduces unpredictable contention. Although existing approaches [10, 12, 13] partition compute, they provide little control over shared resources such as memory bandwidth.

To this end, we propose MuxWise, an LLM serving framework that achieves high goodput across diverse workloads. MuxWise comprises three modules: a *bubble-less multiplex engine*, a *contention-tolerant estimator*, and an *SLO-aware dispatcher*. The engine partitions prefill into layers with negligible overhead, aligning execution latencies for bubble-less multiplexing. The contention-tolerant estimator provides worst-case latency predictions by combining a solo-run predictor with a contention guard derived from one-time offline profiling. Built atop the engine and estimator, the SLO-aware dispatcher schedules diverse LLM requests efficiently by selecting multiplexing plans to maximize goodput.

We implement MuxWise on top of SGLang [52], extending it with PD multiplexing. MuxWise is evaluated extensively on both small and large LLMs using real-world workloads. Experiments show that MuxWise achieves an average 2.20× goodput improvement (up to 3.06×) over state-of-the-art solutions. In summary, our contributions are:

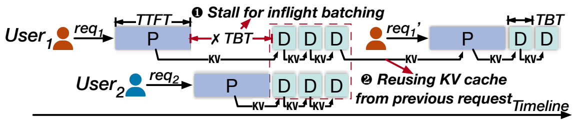


图 1. LLM 服务系统中的典型工作流程：用户 1 发送两个连续的请求，第二个请求重用第一个请求的上下文。用户2发送1个请求。

为了实现服务这些工作负载的高吞吐量，现有的 LLM 服务系统采用了多种优化。图 1 展示了一个典型的工作流程。虽然请求在不同时间到达，但飞行批处理会停止正在进行的解码阶段以预填充新请求，然后在单个批次中一起处理所有解码迭代。它极大地提高了内存密集型解码阶段的计算利用率[47]。由于多轮请求共享上下文，LLM 服务系统通过 KV 缓存池在请求内和请求之间重用中间结果（即 KV 缓存）[26, 52]。

LLM 服务还规定了严格的服务水平目标（SLO）。例如，聊天机器人通常需要 500 毫秒以下的首次令牌时间 (TTFT) 进行预填充，并要求 100 毫秒以下的令牌间隔时间 (TBT) 进行解码。由于 LLM 服务系统中的预填充和解码交织，可能会出现 SLO 违规。在图 1 中，进行中的批处理会停止正在进行的解码。因此，长预填充可能会延迟解码，从而可能违反其 SLO。为了在 SLO 保证的情况下维持高吞吐量峰值吞吐量，现有方法分为两类：分解服务 [32, 53] 和分块预填充 [1]。

至于分解服务，Splitwise [32] 将预填充和解码阶段分为不同的实例以保证 SLO，这有两个缺点。首先，它不能适应服务动态。在 Splitwise 中，GPU 在初始化时静态分配。在波动的请求负载和多样化的服务模式下，通常会导致资源利用不足。其次，由于 KV 缓存池的缩小，它降低了吞吐量。在 GPU 数量相同的情况下，分解为每个实例分配单独的缓存池，从而减少了有效缓存池的大小。这降低了缓存命中率[45]（例如，从 36.6% 降至 4.2%），导致不必要的重新计算和吞吐量下降。此外，虽然LoongServe [44]支持基于请求序列长度和执行阶段的动态GPU分配，但它无法支持跨请求KV缓存重用，从而在多轮工作负载中产生大量的重新计算开销。

分块预填充 [1] 是另一种满足解码 SLO 的方法。它将预填充阶段分成每个 GPU 内的块，并通过解码迭代融合每个块。为了确保计算等效性，每个块都会读取所有先前块生成的 KV 缓存。它通过限制令牌预算来确保解码 SLO，令牌预算定义为来自预填充块和解码批次的新令牌的总和。通过调整块和解码批量大小，它适应服务动态。由于它避免了分解，因此还可以防止因减少 KV 缓存池而造成的有效吞吐量损失。

不幸的是，分块并不是免费的午餐。它在 SLO 合规性和高利用率之间造成了两难境地。由于预填充块和解码迭代必须一起执行，因此令牌预算控制着解码 SLO 的实现和 GPU 饱和度。然而，在实践中找到合理的预算是不可行的。例如，在 8 个 A100 GPU 上部署 70B LLM 需要 4K 预算才能使 GPU 饱和，这比符合 SLO 的预算大 8×（100 毫秒 TBT SLO 为 256）。此外，预填充块中的 TBT 会因预填充块的重复 KV 缓存访问而膨胀。对于多回合工作负载中常见的极长重用上下文，分块预填充甚至可能无法满足 SLO 保证（第 2.3.2 节）。最终，分块预填充无法维持高吞吐量。

实现高产出的 LLM 服务需要更灵活的计算管理。我们提出 GPU 内预填充解码（PD）复用作为一种有前途的新服务范式。具体来说，预填充和解码阶段在 GPU 内的不同流多处理器（SM）上执行。在新范例中，1）计算分区可以以较低的开销重新配置以适应服务动态；2）复用阶段共享GPU内存，保持KV缓存池的高效；3）通过空间共享，预填充和解码独立执行，避免了SLO合规性和利用率之间的权衡。

实现这一范式并非易事。首先，仍然需要阶段协调来启用动态批处理并提高计算利用率。然而，由于预填充和解码延迟差异很大，天真的协调通常会留下 GPU 气泡。其次，空间复用引入了不可预测的竞争。尽管现有的方法[10,12,13]对计算进行分区，但它们对共享资源（例如内存带宽）提供的控制很少。

为此，我们提出了 MuxWise，这是一个 LLM 服务框架，可以在不同的工作负载中实现高吞吐量。 MuxWise 包含三个模块：无气泡多路复用引擎、容争估计器和 SLO 感知调度程序。该引擎将预填充划分为可忽略不计的开销的层，调整执行延迟以实现无气泡复用。容争估计器通过将单独运行的预测器与源自一次性离线分析的争用防护相结合来提供最坏情况的延迟预测。构建在引擎和估计器之上，SLO 感知调度程序通过选择复用计划来有效地调度不同的 LLM 请求，以最大限度地提高吞吐量。

我们在 SGLang [52] 之上实现 MuxWise，并通过 PD 复用对其进行扩展。 MuxWise 使用实际工作负载在小型和大型大语言模型进行了广泛的评估。实验表明，与最先进的解决方案相比，MuxWise 的平均吞吐量提高了 2.20×（高达 3.06×）。总之，我们的贡献是：

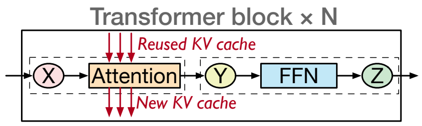


Figure 2. Main architecture of most LLMs.

Table 1. Diverse patterns of typical LLM tasks. Minimum, mean, and maximum values for each metric are reported. The input length includes the length of new and reused context.

	Input length	Output length	Reused length
ShareGPT [4]	4/226/1024	4/195/1838	\
LooGLE [25]	3380/30k/81k	2/15/326	\
OpenThoughts [17]	311/709/4633	684/8374/32k	243
Conversation [34]	891/7538/123k	1/342/2000	0/4496/120k
Tool&agent [34]	891/8596/123k	1/182/2000	0/4905/120k

- We identify key requirements for LLM serving with high goodput through a detailed analysis of prior works.
- We propose a new LLM serving paradigm–PD multiplexing–aligned with these requirements, and present a clean design to effectively serve LLMs with high goodput.
- We evaluate MuxWise under diverse workloads, demonstrating its superiority over state-of-the-art solutions.

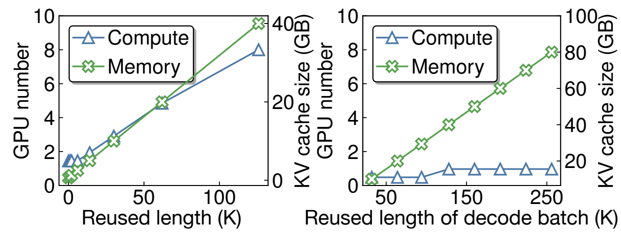
2 Background & Motivation

2.1 LLM Services

Architecture of LLMs. Most LLMs [5, 15, 37, 38] are built upon the transformer architecture [39], with model-specific modifications. Figure 2 illustrates a typical transformer layer, which is replicated multiple times to form an LLM model. Each transformer layer contains an attention layer and a feed-forward network (FFN) layer.

Attention computation requires access to all keys and values from processed tokens and also generates the keys and values of new tokens. To avoid redundant computation, LLM serving systems store this data in a KV cache. In the prefill phase, the KV cache is populated from the requests in previous turns. In each decode iteration, the KV cache is derived from earlier prefill and decode iterations.

Diverse workload patterns. Table 1 illustrates the diverse patterns of five typical LLM tasks. The first three are single-turn requests: ShareGPT [4] is a chatbot task, LooGLE [25] is a long-context understanding task, and OpenThoughts [17] is a reasoning task. LooGLE has a long input length due to long documents. Reasoning often requires long thought processes, so OpenThoughts tends to have a longer output length than others. Requests in OpenThoughts share the same system prompt, which is a constant input context (i.e., reused length in the table). Conversion and Tool&agent [34] are two real-world multi-turn tasks. The output tokens from earlier requests become the input context for later requests in the same session. We use these workloads to conduct experiments that both motivate and evaluate our design.



(a) Prefill phase (b) Decode phase

Figure 3. Required compute and memory for processing different phases under SLO constraints with varied reused context lengths. For prefill (a), the batch size is fixed at 1, the new context length is set to $2K$, and TTFT is set to $400ms$. For decode (b), the batch size is fixed at 32, and TBT is set to $100ms$. These settings are commonly seen in online serving.

2.2 Characterization under SLO constraint

Many prior works [32, 44, 53] have investigated the relationship between resource requirements and SLO attainment concerning input length and batch size. Their experiments show that the prefill phase is compute-intensive, with compute demand growing linearly with input length, while the decode phase is memory-intensive. However, they mainly focus on the simple single-turn case, which does not consider the effect of reused input length.

Under these circumstances, we further study how the reused length impacts the compute and memory demands of prefill and decode. In our experiment, the reused length spans the range shown in Table 1, and LlaMA-70B [15] is deployed with tensor parallelism [51] on a server with 8 A100 GPUs. All GPUs are configured with the same partial compute resource, defined by the SM number. For each reused length, we determine the best-fit GPU partition ratio (denoted as GPU_{ratio}) to satisfy the SLO target. Figure 3 reports the total compute demand of LlaMA-70B under different reused lengths, computed as $GPU_{num} = GPU_{ratio} \times 8$.

As shown in Figure 3-(a), prefill phase requires increasingly more compute resources to meet SLO targets as the reused length grows. In contrast, the compute demand of the decode phase shows less sensitivity. Thus, it is also critical to allocate more compute to the prefill phase as the reused length increases. Further, the distinct compute requirements of two phases necessitate a runtime compute resource partition for SLO attainment and high utilization.

Figure 3-(b) shows that the KV cache required by both the prefill and decode easily reaches tens or even hundreds of gigabytes. This is common in multi-turn LLM services, which produce ultra-long reused contexts. It is preferable to keep the KV cache in the same memory space (aggregated serving) for efficient reuse across phases and requests.

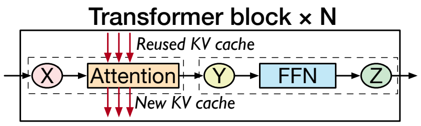


图 2. 大多数大语言模型要架构。

表 1. 典型 LLM 任务的不同模式。报告每个指标的最小值、平均值和最大值。输入长度包括新的和重用的上下文的长度。

	Input length	Output length	Reused length
ShareGPT [4]	4/226/1024	4/195/1838	\
LooGLE [25]	3380/30k/81k	2/15/326	\
OpenThoughts [17]	311/709/4633	684/8374/32k	243
Conversation [34]	891/7538/123k	1/342/2000	0/4496/120k
Tool&agent [34]	891/8596/123k	1/182/2000	0/4905/120k

- 我们通过对先前工作的详细分析，确定了高产出的法学硕士的关键要求。
- 我们提出了一种新的 LLM 服务范式——PD 多路复用——符合这些要求，并提出了一种简洁的设计，以有效地为 LLM 提供高吞吐量的服务。
- 我们在不同的工作负载下评估 MuxWise，证明其相对于最先进的解决方案的优越性。

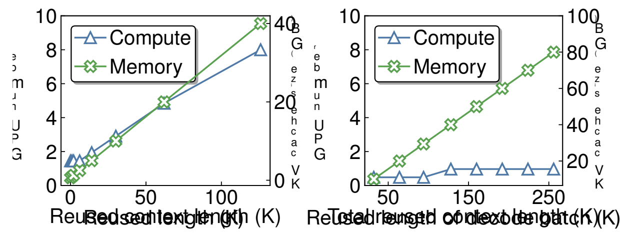
2 背景与动机

2.1 大语言模型

大语言模型构。大多数 LLM [5,15,37,38] 都是基于 Transformer 架构 [39] 构建的，并进行了特定于模型的修改。图 2 展示了一个典型的 Transformer 层，该层被复制多次以形成 LLM 模型。每个变压器层包含一个注意力层和一个前馈网络（FFN）层。

注意力计算需要访问已处理令牌中的所有键和值，并生成新令牌的键和值。为了避免冗余计算，LLM 服务系统将此数据存储在 KV 缓存中。在预填充阶段，KV 缓存由前几轮的请求填充。在每次解码迭代中，KV 缓存源自早期的预填充和解码迭代。

不同的工作负载模式。表 1 说明了五种典型的法学硕士任务的不同模式。前三个是单轮请求：ShareGPT [4] 是一个聊天机器人任务，LooGLE [25] 是一个长上下文理解任务，OpenThoughts [17] 是一个推理任务。由于文档较长，LooGLE 的输入长度也较长。推理通常需要较长的思维过程，因此 OpenThoughts 的输出长度往往比其他方法更长。 OpenThoughts 中的请求共享相同的系统提示，这是一个恒定的输入上下文（即表中重用的长度）。转换和工具与代理[34]是两个现实世界的多回合任务。早期请求的输出标记将成为后续请求的输入上下文



(a) Prefill phase (b) Decode phase

图 3. 在具有不同重用上下文长度的 SLO 约束下处理不同阶段所需的计算和内存。对于预填充 (a)，批量大小固定为 1，新上下文长度设置为 $2K$，TTFT 设置为 400ms 。对于解码(b)，批量大小固定为32，TBT设置为100ms 。这些设置在在线服务中很常见。

在同一个会话中。我们使用这些工作负载来进行实验，以激励和评估我们的设计。

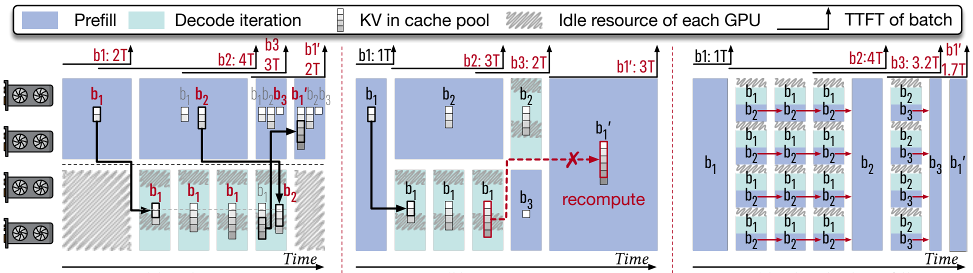
2.2 SLO约束下的表征

许多先前的工作 [32,44,53] 研究了资源需求和 SLO 达到（有关输入长度和批量大小）之间的关系。他们的实验表明，预填充阶段是计算密集型的，计算需求随着输入长度线性增长，而解码阶段是内存密集型的。然而，他们主要关注简单的单匝情况，没有考虑重用输入长度的影响。

在这种情况下，我们进一步研究重用长度如何影响预填充和解码的计算和内存需求。在我们的实验中，重用的长度跨越表1所示的范围，并且LlaMA-70B [15]以张量并行性[51]部署在具有8个A100 GPU的服务器上。所有 GPU 都配置有相同的部分计算资源，由 SM 编号定义。对于每个重用的长度，我们确定最适合的 GPU 分区比率（表示为 GPU_{ratio} ）以满足 SLO 目标。图 3 报告了 LlaMA-70B 在不同重用长度下的总计算需求，计算公式为 $GPU_{num} = GPU_{ratio} \times 8$。

如图 3-(a) 所示，随着重用长度的增长，预填充阶段需要越来越多的计算资源来满足 SLO 目标。相反，解码阶段的计算需求表现出较低的敏感性。因此，随着重用长度的增加，为预填充阶段分配更多计算也很重要。此外，两个阶段不同的计算要求需要运行时计算资源分区，以实现 SLO 和高利用率。

图3-(b)显示预填充和解码所需的KV缓存很容易达到数十甚至数百GB。这在多轮 LLM 服务中很常见，这些服务会产生超长的重用上下文。最好将 KV 缓存保留在相同的内存空间（聚合服务）中，以便跨阶段和请求高效重用。



(a) Splitwise (b) LoongServe (c) Chunked-prefill

Figure 4. Processing four LLM request batches on 4 GPUs using (a) Splitwise, (b) LoongServe (c) chunked-prefill. All methods satisfy the TBT SLO (T per decode iteration). Specifically, b_1 arrives at $0T$, b_2 at $1T$, b_3 at $3T$, and b_1' at $5T$. b_1' denotes a subsequent request batch that reuses the KV cache of b_1. Inefficient TTFTs are marked in red for each method. KV cache management is shown only for the two disaggregated methods, as they require migration or recomputation. In (a) and (b), solid black arrows represent migration, while dashed red arrows with cross markers denote recomputation. In (a), the KV cache column with a red b_i indicates the active batch. In (c), the red arrow denotes KV cache reads from earlier chunks.

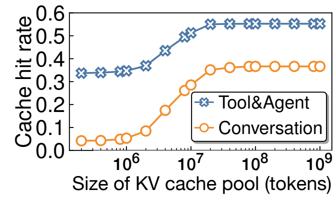
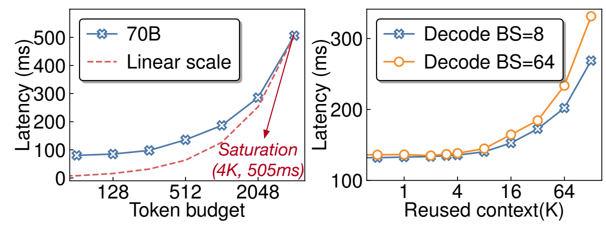


Figure 5. Cache hit rates under varying capacities of the KV cache pool. The eviction policy is Least Recently Used. For serving a 70B LLM, achieving the optimal hit rate requires 3.3 TB of memory. Workload trace details are shown in Table 1.

In a nutshell, we make two observations: *1) Appropriate and dynamic compute partition is essential for meeting the distinct SLO targets of different phases under diverse workloads. 2) Reusing the KV cache across phases and requests is critical for reducing redundant computation and improving goodput.*



(a) Sweet token budget (b) Chunk with reused context

Figure 6. (a) Sweet spot of the token budget in chunk-prefill. The decode uses a fixed batch size of 32, with each request having a reused context length of 1K tokens. (b) Latencies with varied reused context of the fused prefill chunk in chunk-prefill. The token budget is fixed at 512, and the reused context length of decode phase is the same as in (a).

2.3 Deficiencies of Existing Works

2.3.1 Disaggregated Serving. Disaggregating approaches partition GPUs across phases to meet the SLO targets in LLM serving and can be further divided into static and dynamic disaggregation methods. Figure 4-(a) illustrates the static approach (Splitwise [32]), while Figure 4-(b) shows the dynamic approach (LoongServe [44]).

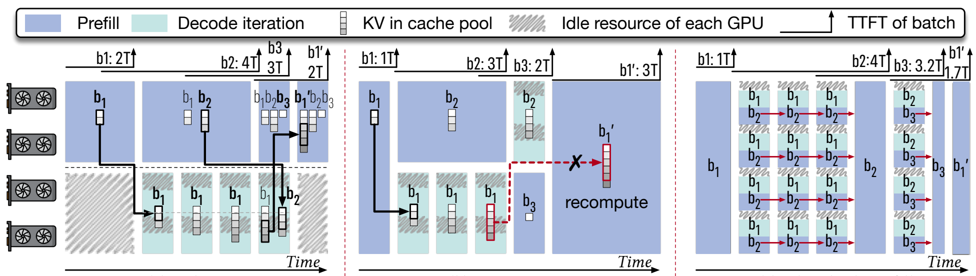
Static disaggregation. As shown in Figure 4-(a), there is a prefill instance and a decode instance with Splitwise [32]. Each instance occupies two GPUs statically and has its own KV cache pool. The GPU number is static after the instance is initialized. In this case, Splitwise suffers from two problems.

First, Splitwise does not adapt to serving dynamics. For example, when batch b1 arrives, only two GPUs process the prefill while the other two GPUs for decoding remain idle. In online serving, such idle periods are common as request loads fluctuate. *Second, the coupled management of compute and memory introduces further inefficiencies.* For instance, if the decode phase of b1 in Figure 4-(a) requires two GPUs to store the KV cache, the system must also allocate two GPUs for computation. Since compute and memory requirements are misaligned, as shown in Figure 3-(b), the GPUs' compute resources may be underutilized.

In addition, each instance must maintain its own model weights and KV cache pool. As a result, the KV cache pool in Figure 4-(a) is at most half the size of that with four GPUs under non-disaggregated execution. Furthermore, experimental results in Figure 5 show that this reduced capacity sharply lowers the KV cache hit rate in multi-turn workloads, ultimately degrading the system's goodput.

Dynamic disaggregation. LoongServe [44] supports dynamic GPU partitioning across the two phases. Specifically, it scales GPU resources based on the sequence length and execution phase. As shown in Figure 4-(b), when batch b1 arrives, the scheduler assigns four GPUs to prefill. After prefill, it scales down to two GPUs for the decode iterations.

However, LoongServe still causes idleness due to coupled management, and worse, it trades KV cache reuse for adaptiveness needed in serving dynamics. To avoid duplication, it immediately releases the KV cache on original GPUs. Thus, KV caches are reused only from prefill to decode within a single request and cannot be reused across multi-turn requests. In Figure 4-(b), when b1' needs to reuse the KV cache generated by b1, LoongServe recomputes the entire KV cache.



(a) Splitwise (b) LoongServe (c) Chunked-prefill

图 4. 使用 (a) Splitwise、(b) LoongServe (c) 分块预填充在 4 个 GPU 上处理四个 LLM 请求批次。所有方法都满足 TBT SLO（每次解码迭代的 T）。具体来说，b_1 到达 $0T$，b_2 到达 $1T$，b_3 到达 $3T$，b'_1 到达 $5T$。b'_1 表示后续请求批次，重用 b_1 的 KV 缓存。每种方法的低效 TTFT 均标记为红色。仅针对两种分解方法显示了 KV 缓存管理，因为它们需要迁移或重新计算。在（a）和（b）中，黑色实线箭头表示迁移，而带有十字标记的红色虚线箭头表示重新计算。在 (a) 中，带有红色 b_i 的 KV 缓存列表示活动批次。在 (c) 中，红色箭头表示 KV 缓存从较早的块读取。

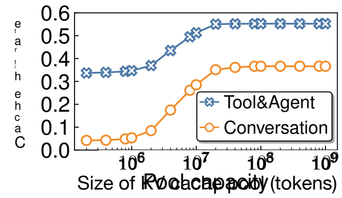


图 5. KV 缓存池不同容量下的缓存命中率。驱逐政策是最近最少使用的。为了服务于 70B LLM，达到最佳命中率需要 3.3 TB 内存。工作负载跟踪详细信息如表 1 所示。

简而言之，我们有两个观察结果：1）适当的动态计算分区对于满足不同工作负载下不同阶段的不同 SLO 目标至关重要。 2）跨阶段和请求重用KV缓存对于减少冗余计算和提高吞吐量至关重要。

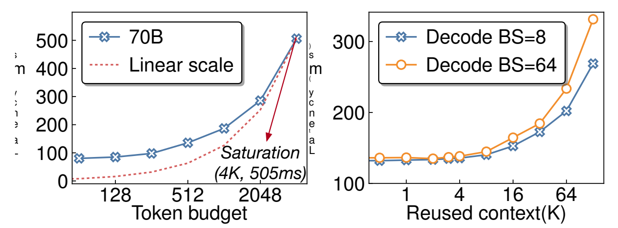
2.3 现有工作的不足

2.3.1 分类服务。分解方法跨阶段划分 GPU，以满足 LLM 服务中的 SLO 目标，并且可以进一步分为静态和动态分解方法。图 4-(a) 说明了静态方法（Splitwise [32]），而图 4-(b) 显示了动态方法（LoongServe [44]）。

静态分解。如图 4-(a) 所示，有一个预填充实例和一个带有 Splitwise 的解码实例 [32]。每个实例静态占用两个GPU，并拥有自己的KV缓存池。实例初始化后，GPU编号是静态的。在这种情况下，Splitwise 面临两个问题。

首先，Splitwise 不适应服务动态。例如，当b1批次到达时，只有两个GPU处理预填充，而另外两个用于解码的GPU保持空闲。在在线服务中，随着请求负载的波动，这种空闲期很常见。其次，计算和内存的耦合管理进一步降低了效率。例如，如果图4-(a)中的b1的解码阶段需要两个GPU来存储KV缓存，则系统还必须分配两个GPU来进行计算。由于计算和内存需求不一致，如图 3-(b) 所示，GPU 的计算资源可能未得到充分利用。

此外，每个实例必须维护自己的模型权重和KV缓存池。因此，图 4-(a) 中的 KV 缓存池大小至多是非分解执行下四个 GPU 的 KV 缓存池大小的一半。此外，图 5 中的实验结果表明，这种容量减少



(a) Sweet token budget (b) Chunk with reused context

图 6.(a) 块预填充中代币预算的最佳点。解码使用固定批量大小 32，每个请求具有 1K 令牌的重用上下文长度。 (b) 块预填充中融合预填充块的不同重用上下文的延迟。令牌预算固定为512，解码阶段的重用上下文长度与(a)中相同。

急剧降低多轮工作负载中的 KV 缓存命中率，最终降低系统的吞吐量。

动态分解。 LoongServe [44] 支持跨两个阶段的动态 GPU 分区。具体来说，它根据序列长度和执行阶段来扩展 GPU 资源。如图 4-(b) 所示，当批次 b1 到达时，调度程序分配 4 个 GPU 进行预填充。预填充后，它会缩减至两个 GPU 来进行解码迭代。

然而，LoongServe 仍然会由于耦合管理而导致空闲，更糟糕的是，它以 KV 缓存重用来换取服务动态所需的自适应性。为了避免重复，它会立即释放原始GPU上的KV缓存。因此，KV 缓存仅在单个请求中从预填充到解码的过程中被重用，而不能在多轮请求中重用。在图4-(b)中，当b1’需要重用b1生成的KV缓存时，LoongServe会重新计算整个KV缓存。

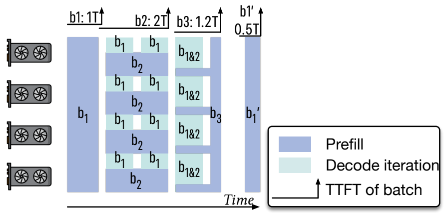


Figure 7. An ideal solution: prefill-decode multiplexing.

2.3.2 Chunked-prefill.

Chunked-prefill [1] adopts intra-GPU compute fusion. As shown in Figure 4-(c), it splits prefill into chunks and fuses each chunk with a decode iteration. To guarantee decode SLOs, chunked-prefill caps the token budget, which is the sum of new tokens from the prefill chunk and the decode batch. While chunked-prefill has known drawbacks such as quadratic memory overhead [53], we find another drawback. Specifically, chunking introduces a dilemma between SLO attainment and utilization.

Figure 6-(a) presents TBT in Chunked-prefill of varying token budget. In this experiment, the decode iteration for fusion has a static batch size of 32 and a reused context length of 1K tokens, and Llama3-70B is deployed on a server with 8 A100 GPUs. As shown, the latency does not increase linearly with the token budget until it reaches $4K$. This indicates that saturating the GPUs requires a prefill chunk with input length of $(4K - 32)$. However, the corresponding latency is $505ms$, far above the typical TBT SLO target ($< 100ms$).

Figure 6-(b) presents TBT in Chunked-prefill with varying reused context lengths of the prefill. In this experiment, the token budget is fixed at 512, and the reused context length of decode iteration is 1K. As shown, TBT increases noticeably after the reused context exceeds $4K$. This reused context length is common in long-context understanding and multi-turn workloads, as shown in Table 1. In such cases, Chunked-prefill easily leads to SLO violations.

2.4 New Paradigm & Challenges

As shown in Figure 7, we propose an intra-GPU prefill-decode (PD) multiplexing paradigm to overcome the above limitations. Specifically, prefill and decode dynamically share the compute resources (SMs) within each GPU. By reserving sufficient SMs to satisfy decode SLOs and assigning the remaining SMs to prefill, high-goodput LLM serving is achieved. PD multiplexing overcomes the limitations of prior methods, benefiting from the following abilities.

First, multiplexing enables dynamic and adaptive compute management. As shown, compute resources can be flexibly allocated between the two phases to maximize system goodput while guaranteeing SLOs. Second, multiplexing decouples compute from memory management. Although the two phases partition compute resources, they share the memory space on each GPU, enabling efficient KV cache reuse. Third, multiplexing allows prefill and decode to run independently without stalling one another, avoiding the dilemma between SLO attainment and system goodput.

However, integrating intra-GPU multiplexing into existing LLM serving systems is non-trivial. There are two challenges to realizing this paradigm. **C-1: GPU bubbles from naive integration.** Current systems have frequent prefill–decode interactions due to the inflight batching mechanism. One phase can easily block the other, creating GPU bubbles. **C-2: Unmanaged contention in spatial multiplexing.** Existing techniques [10, 12, 13] partition only SMs while leaving memory bandwidth unmanaged. As a result, memory bandwidth contention can lead to SLO violations.

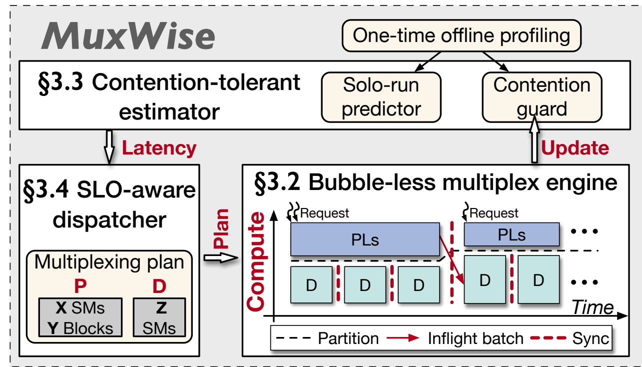


Figure 8. Architecture overview of MuxWise.

3 MuxWise's Design

3.1 Architecture Overview

Based on the PD multiplexing paradigm, we propose MuxWise, an LLM serving framework that achieves high goodput on diverse workloads. Figure 8 shows the overview of MuxWise that comprises (1) a bubble-less multiplex engine, (2) a contention-tolerant estimator, and (3) an SLO-aware dispatcher.

To enable PD multiplexing with bubble-less coordination, the engine partitions prefill into layer-wise execution, aligning its latency with decode execution. Notably, layer-wise execution incurs negligible overhead and avoids the inefficiencies of chunk-prefill. In addition, it provides an extra benefit: preempting ultra-long prefills to prevent the SLO violations caused by them.

To avoid SLO violations caused by unpredictable contention, the estimator provides worst-case latency estimates by combining a solo-run latency predictor with a contention guard. Both components are built from one-time offline profiling for each LLM on a given hardware. They consider five key factors: reused length, input length, output length, decoding batch size, and partition configuration. LLM-specific factors are extracted from workload traces to guide profiling.

As requests arrive, the SLO-aware dispatcher leverages the engine and estimator to schedule prefill layers and decode iterations dynamically for high goodput. Specifically, the

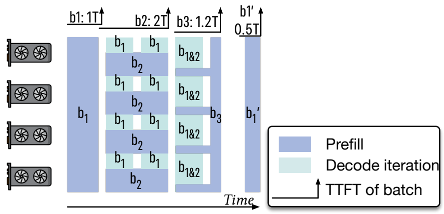


图 7. 理想的解决方案：预填充解码复用。

2.3.2 分块预填充。 Chunked-prefill [1]采用GPU内计算融合。如图 4-(c) 所示，它将预填充拆分为块，并通过解码迭代融合每个块。为了保证解码 SLO，分块预填充限制了令牌预算，该预算是来自预填充块和解码批次的新令牌的总和。虽然分块预填充具有已知的缺点，例如二次内存开销 [53]，但我们发现了另一个缺点。具体来说，分块在 SLO 实现和利用率之间引入了困境。

图 6-(a) 展示了不同代币预算的分块预填充中的 TBT。在本实验中，融合的解码迭代的静态批量大小为 32，重用上下文长度为 1K 个令牌，Llama3-70B 部署在具有 8 个 A100 GPU 的服务器上。如图所示，延迟不会随着代币预算线性增加，直到达到 $4K$。这表明要使 GPU 饱和需要输入长度为 $(4K-32)$ 的预填充块。然而，相应的延迟为 $505ms$，远高于典型的 TBT SLO 目标 (< 100 ms)。

图 6-(b) 展示了分块预填充中的 TBT，其中预填充的重用上下文长度不同。在本实验中，令牌预算固定为512，解码迭代的重用上下文长度为1K。如图所示，在重用上下文超过 $4K$ 后，TBT 显着增加。这种重用的上下文长度在长上下文理解和多轮工作负载中很常见，如表 1 所示。在这种情况下，分块预填充很容易导致 SLO 违规。

2.4 新范式与挑战

如图 7 所示，我们提出了一种 GPU 内预填充解码 (PD) 复用范例来克服上述限制。具体来说，预填充和解码动态共享每个 GPU 内的计算资源 (SM)。通过保留足够的 SM 来满足解码 SLO 并将剩余的 SM 分配给预填充，可以实现高吞吐量的 LLM 服务。 PD 复用克服了现有方法的局限性，并受益于以下功能。

首先，多路复用支持动态和自适应计算管理。如图所示，计算资源可以在两个阶段之间灵活分配，以最大限度地提高系统吞吐量，同时保证 SLO。其次，多路复用将计算与内存管理解耦。尽管这两个阶段划分了计算资源，但它们共享每个 GPU 上的内存空间，从而实现高效的 KV 缓存重用。第三，多路复用允许预填充和解码独立运行

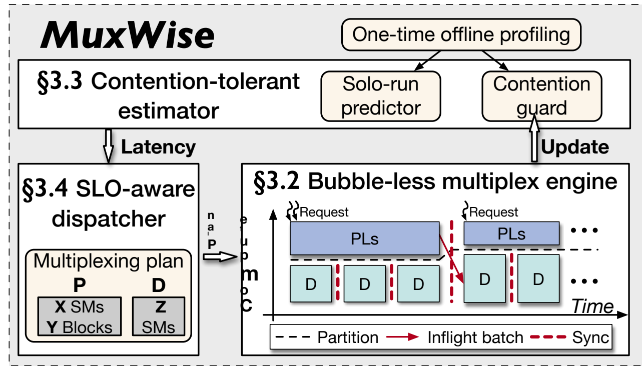


图 8.MuxWise 的架构概述。

不会互相拖延，避免 SLO 达到和系统良好吞吐量之间的困境。

然而，将 GPU 内多路复用集成到现有的 LLM 服务系统中并非易事。实现这一范式面临两个挑战。 C-1：GPU 因简单的集成而出现泡沫。由于飞行批处理机制，当前系统具有频繁的预填充-解码交互。一相很容易阻塞另一相，从而产生 GPU 气泡。 C-2：空间复用中不受管理的争用。现有技术 [10,12,13] 仅对 SM 进行分区，同时使内存带宽不受管理。因此，内存带宽争用可能会导致 SLO 违规。

3 MuxWise 的设计

3.1 架构概述

基于 PD 多路复用范式，我们提出了 MuxWise，这是一种 LLM 服务框架，可以在不同的工作负载上实现高吞吐量。图 8 显示了 MuxWise 的概述，它包括 (1) 无气泡多路复用引擎、(2) 容争估计器和 (3) SLO 感知调度程序。

为了实现具有无气泡协调的 PD 多路复用，引擎将预填充划分为分层执行，使其延迟与解码执行保持一致。值得注意的是，逐层执行带来的开销可以忽略不计，并避免了块预填充的低效率。此外，它还提供了一个额外的好处：抢占超长预填充，以防止它们引起的 SLO 违规。

为了避免由于不可预测的争用而导致 SLO 违规，估计器通过将单独运行的延迟预测器与争用防护相结合来提供最坏情况的延迟估计。这两个组件都是根据给定硬件上每个大语言模型次性离线分析构建的。他们考虑了五个关键因素：重用长度、输入长度、输出长度、解码批量大小和分区配置。从工作负载跟踪中提取 LLM 特定因素以指导分析。

当请求到达时，SLO 感知调度程序利用引擎和估计器来调度预填充层并动态解码迭代以获得高吞吐量。具体来说，

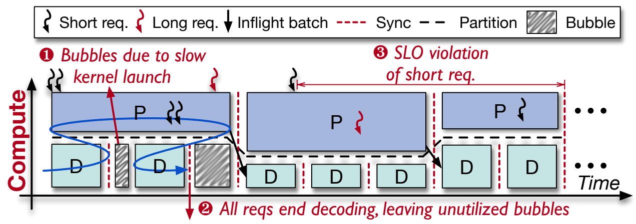


Figure 9. Bubbles and SLO violation of naive intra-process multiplexing. → represents the kernel launch order.

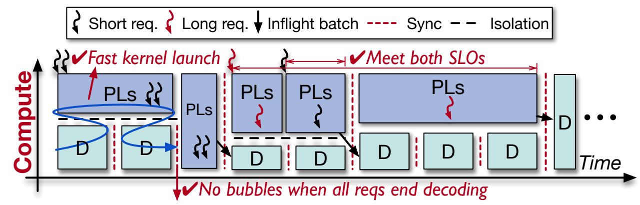


Figure 10. Bubble-less coordination using layer-wise scheduling for the prefill phase and graph-level scheduling for the decode phase. PLs is the short for prefill layers.

dispatcher reserves best-fit SMs to satisfy decode SLOs based on the worst-case estimation, and assigns the remaining SMs to prefill. Meantime, during online serving, it further refines the contention guard using runtime execution data.

3.2 Bubble-less Multiplex Engine

3.2.1 Spatial Multiplexing Technique. According to the analysis in §2.4, MuxWise imposes two requirements for PD multiplexing. First, the compute resources must be dynamically partitioned between the two phases with low overhead. Second, the memory space must be shared across the phases to enable efficient KV cache reuse. To meet these requirements, we examine existing approaches for spatially partitioning GPU compute across tasks.

We categorize these approaches into two types: inter- and intra-process partitioning. Inter-process approaches, such as CUDA MIG [12] and CUDA MPS [13], cannot provide flexible compute resource adjustment, let alone the introduced cross-process communication between prefill and decode. In contrast, the intra-process approach GreenContext [10] enables low-overhead resource adjustment by binding CUDA streams to specific SMs, with reconfiguration costing only a stream synchronization (on the order of microseconds). Furthermore, because both phases reside in the same process under GreenContext, they can directly share the same memory space for maintaining a single KV cache pool.

3.2.2 Inefficiencies from Naive Integration. With intra-process compute partitioning, a naive way to support PD multiplexing is to launch the ongoing decode iteration before the prefill phase of new request. This ordering is motivated by launch latency difference: launching a decode iteration takes less than 0.5 ms, whereas launching a prefill phase takes tens of milliseconds.

Ideally, the launch latency of either a prefill phase or decode iteration can be reduced to a single CUDA graph launch (~0.5*ms*). However, CUDA graph requires offline construction with static configuration and incurs memory overhead. In prefill phase, both batch size and input length vary, whereas decode iteration varies only in batch size. Thus, single-graph optimization is feasible only for decode phase with several selected batch sizes [40], while applying it to prefill phase would require capturing much more graphs, incurring unacceptable memory overhead.

Prefill phase can be optimized through piecewise CUDA graph [40], which splits the prefill phase into multiple layer-wise CUDA graphs. It still incurs ~**10 ms** launch overhead for Llama-70B on 8 A100 GPUs. Fortunately, prefill phases consists of long-duration kernels, which are typically longer than the launch time. It does not suffer noticeable performance degradation from launch overhead in most cases.

To this end, when both a prefill phase and a decode iteration are pending, MuxWise prioritizes launching the decode iteration; otherwise, the SMs allocated for the decode phase would remain idle for tens of milliseconds. However, this naive approach still introduces two types of GPU bubbles and can also lead to SLO violations.

Firstly, when the prefill launch time exceeds the execution time of a decode iteration, next decode iteration cannot launch in time, and GPU bubbles occur. As shown on the left of Figure 9, a bubble appears between two decode iterations because the serving system must return newly generated tokens after each iteration.

Secondly, bubbles can arise from unpredictable termination of decode. As depicted in the middle of Figure 9, all requests in a decode batch may finish token generation while a concurrent prefill phase has already been launched. Due to the non-preemptive nature of GPU execution, the launched prefill cannot be interrupted to reclaim compute resources.

Thirdly, SLO violations may occur due to workload skew among requests, as illustrated in the upper-right of Figure 9. Context lengths can vary significantly, with short conversations coexisting alongside long-text summarizations. In such cases, a short request may suffer long queuing delays while waiting for the prefill of an ultra-long request. If the short request has limited SLO slack, it is likely to miss its deadline.

3.2.3 Bubble-less Coordination. The above inefficiencies stem from the large latency discrepancy between the prefill and decode phases. This is because prefill phases typically take longer to launch and execute, and the execution time of both phases is highly variable at runtime. To address this, we propose layer-wise execution for prefill and query-based synchronization.

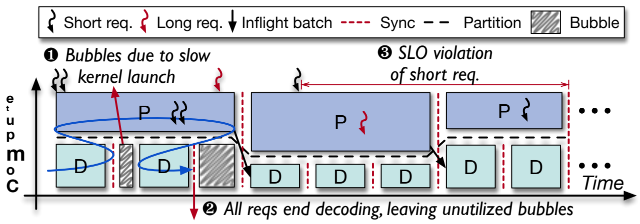


图 9. 原生进程内多路复用的气泡和 SLO 违规。 → 代表内核启动顺序。

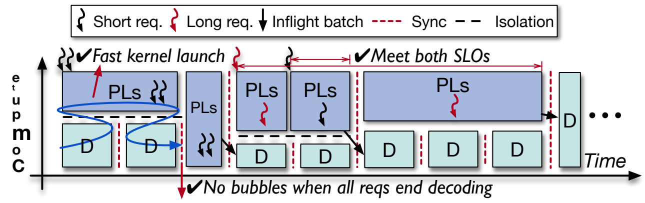


图 10. 在预填充阶段使用分层调度，在解码阶段使用图形级调度的无气泡协调。 PLs 是预填充层的缩写。

调度程序保留最适合的 SM 以满足基于最坏情况估计的解码 SLO，并分配剩余的 SM 进行预填充。同时，在在线服务期间，它使用运行时执行数据进一步细化争用防护。

3.2 无气泡多重引擎

3.2.1 空间复用技术。根据§2.4中的分析，MuxWise对PD复用提出了两个要求。首先，计算资源必须以较低的开销在两个阶段之间动态分配。其次，内存空间必须在各个阶段之间共享，以实现高效的 KV 缓存重用。为了满足这些要求，我们研究了跨任务对 GPU 计算进行空间分区的现有方法。

我们将这些方法分为两种类型：进程间分区和进程内分区。诸如CUDA MIG [12]和CUDA MPS [13]之类的进程间方法无法提供灵活的计算资源调整，更不用说在预填充和解码之间引入跨进程通信了。相比之下，进程内方法 GreenContext [10] 通过将 CUDA 流绑定到特定的 SM 来实现低开销的资源调整，并且重新配置仅花费流同步（大约微秒）。此外，由于两个阶段都驻留在 GreenContext 下的同一进程中，因此它们可以直接共享相同的内存空间来维护单个 KV 缓存池。

3.2.2 简单集成导致的低效率。通过进程内计算分区，支持 PD 复用的一种简单方法是在新请求的预填充阶段之前启动正在进行的解码迭代。这种排序是由启动延迟差异驱动的：启动解码迭代需要不到 0.5 毫秒，而启动预填充阶段需要数十毫秒。

理想情况下，预填充阶段或解码迭代的启动延迟可以减少到单个 CUDA 图启动 (~0.5ms)。然而，CUDA 图需要静态配置的离线构建，并且会产生内存开销。在预填充阶段，批量大小和输入长度都变化，而解码迭代仅批量大小变化。因此，单图优化仅适用于具有多个选定批量大小的解码阶段[40]，而将其应用于预填充阶段需要捕获更多的图形，从而产生不可接受的内存开销。

预填充阶段可以通过分段 CUDA 图 [40] 进行优化，它将预填充阶段分成多个分层 CUDA 图。 Llama-70B 在 8 个 A100 GPU 上仍然会产生 ~10 毫秒的启动开销。幸运的是，预填充阶段由持续时间较长的内核组成，通常比启动时间长。在大多数情况下，它不会因启动开销而遭受明显的性能下降。

为此，当预填充阶段和解码迭代都处于待处理状态时，MuxWise 会优先启动解码迭代；否则，分配给解码阶段的 SM 将保持空闲数十毫秒。然而，这种幼稚的方法仍然引入了两种类型的 GPU 气泡，并且还可能导致 SLO 违规。

首先，当预填充启动时间超过解码迭代的执行时间时，下一次解码迭代无法及时启动，并且会出现GPU气泡。如图 9 左侧所示，两次解码迭代之间会出现气泡，因为服务系统必须在每次迭代后返回新生成的令牌。

其次，不可预测的解码终止可能会产生气泡。如图 9 中间所示，解码批次中的所有请求都可能完成令牌生成，同时并发预填充阶段已经启动。由于 GPU 执行的非抢占性质，启动的预填充不能被中断以回收计算资源。

第三，由于请求之间的工作负载偏差，可能会发生 SLO 违规，如图 9 的右上角所示。上下文长度可能会有很大差异，短对话与长文本摘要共存。在这种情况下，短请求在等待超长请求的预填充时可能会遭受长时间的排队延迟。如果短期请求的 SLO 余裕有限，则很可能会错过截止日期。

3.2.3 无气泡协调。上述低效率源于预填充和解码阶段之间的巨大延迟差异。这是因为预填充阶段通常需要更长的时间来启动和执行，并且两个阶段的执行时间在运行时变化很大。为了解决这个问题，我们建议分层执行预填充和基于查询的同步。

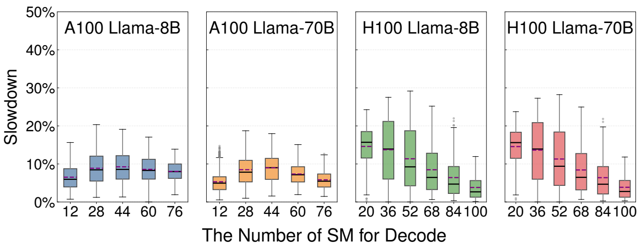


Figure 11. Slowdowns in decode due to contention with different multiplexing configurations of prefill and decode across models and GPUs.

Layer-wise execution for prefill. As shown in Figure 10, MuxWise splits the prefill phase into layers (PLs). Based on this new granularity, MuxWise eliminates GPU bubbles and prevents SLO violations. For the first type of bubble, MuxWise can launch enough PBs to occupy compute resources for prefill, and return in time before the decode phase finishes computation. For the second type, MuxWise switches the execution of later prefill layers into a new GreenContext, just after the decode phase terminates. For SLO violations caused by long requests, layer-wise execution enables preemption, allowing short requests to be prioritized, thereby meeting the SLO targets of both. Importantly, layer-wise execution incurs negligible overhead, since LLMs are inherently structured as multiple transformer layers.

Query-based synchronization. In addition to the above bubbles, inflight batching can also introduce GPU bubbles. Specifically, when the last prefill layer completes, it must block the next decode iteration to merge requests into the decode batch. This blocking creates small bubbles, since prefill and decode rarely finish simultaneously. To address this, MuxWise employs query-based synchronization that periodically polls CUDA events. MuxWise continues launching decode batches and prefill layers asynchronously, and when an event is observed complete, the corresponding prefill request is immediately merged into the current decode batch.

3.3 Contention-tolerant Estimator

When the prefill and decode phases are spatially multiplexed, contention can arise, particularly from unmanaged resources such as memory bandwidth. We begin by analyzing contention between the two phases under spatial multiplexing and then introduce our modeling method.

3.3.1 Contention Analysis. While GreenContext supports precise compute resource allocation, it cannot manage memory or network bandwidth. In particular, efficient techniques for bandwidth management are lacking. Worse, current GPUs do not expose runtime monitoring of bandwidth usage, and both prefill and decode phases can heavily consume bandwidth, making contention hard to predict.

Table 2. Compute analysis for prefill and decode phases.

	Attention	**FFN**
Prefill w/o cache	$O(Ld^2 + L^2d)$	$O(Ld^2)$
Prefill w/ cache	$O(nd^2 + Lnd)$	$O(nd^2)$
Decode	$O(d^2 + (r+1)d)$	$O(d^2)$

To evaluate the impact on execution slowdown, we extensively profile prefill and decode under multiplexing using Llama-8B and Llama-70B. Figure 11 reports the decode slowdown on servers with 8 A100 and 8 H100 GPUs. The x-axis denotes the number of SMs allocated to the decode batch, with the remaining SMs assigned to the prefill batch. For each configuration, the prefill batch's total context length (reused + new) ranges from 1,024 to 128K tokens, whereas the decode batch's reused length ranges from 1,024 to 1,024K tokens. This profiling takes one week.

As shown in Figure 11, contention-induced slowdown ranges from nearly zero to about 30% across different partition configurations and GPUs. The high variation across hardware partitions indicates the inherent unpredictability of contention slowdown. Although both models exhibit similar slowdown trends on the same GPUs due to their architectural similarity, this observation does not aid contention modeling. Meanwhile, results for prefill are similar but omitted due to space constraints.

3.3.2 Worst-case Estimation for SLO guarantee. To mitigate the risk of SLO violations caused by unpredictable contention in online serving, MuxWise introduces a worst-case latency estimation method tailored for SLO guarantees. *The key observation is that precise latency prediction is not the only way to guarantee SLO. What matters is ensuring that the latency of a scheduled phase, given its allocated compute resources, does not exceed the predefined target.* Thus, MuxWise performs worst-case estimation by first predicting its solo-run latency, and then applying a maximum slowdown factor.

Solo-run predictor. To predict solo-run latency, we analyze the compute complexity of prefill and decode, and construct a predictor using offline profiling. The latency of each prefill or decode iteration is determined by the token lengths of the reused and new context. Table 2 summarizes the compute analysis of the prefill and decode phases with a batch size of 1. The key factors are as follows:

- d: The hidden dimension of each token's representation.
- L: The total token length.
- r: The token length of the reused (cached) context.
- $n = L - r$: The token length of the new context.

Based on the complexity analysis in Table 2, we build latency prediction models for the prefill and decode phases. The prefill model is given in Equation 1, and the decode model in Equation 2, where all θ terms are coefficients. We

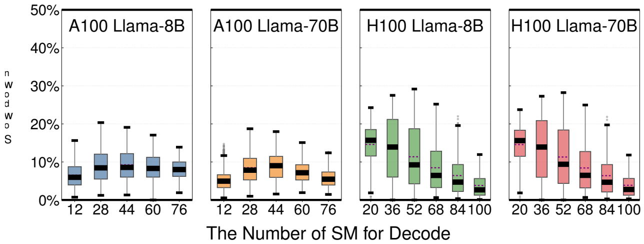


图 11. 由于跨模型和 GPU 的预填充和解码的不同复用配置的争用，导致解码速度减慢。

预填充的分层执行。如图 10 所示，MuxWise 将预填充阶段分为层 (PL)。基于这种新的粒度，MuxWise 消除了 GPU 气泡并防止 SLO 违规。对于第一种类型的气泡，MuxWise 可以启动足够的 PB 来占用计算资源进行预填充，并在解码阶段完成计算之前及时返回。对于第二种类型，MuxWise 在解码阶段终止后将后面的预填充层的执行切换到新的 GreenContext 中。对于长请求引起的 SLO 违规，分层执行可以实现抢占，允许对短请求进行优先级排序，从而满足两者的 SLO 目标。重要的是，分层执行带来的开销可以忽略不计，因为 LLM 本质上是由多个转换器层构成的。

基于查询的同步。除了上述气泡之外，运行中批处理还会引入 GPU 气泡。具体来说，当最后一个预填充层完成时，它必须阻止下一次解码迭代，以将请求合并到解码批次中。这种阻塞会产生小气泡，因为预填充和解码很少同时完成。为了解决这个问题，MuxWise 采用基于查询的同步来定期轮询 CUDA 事件。 MuxWise 继续异步启动解码批次和预填充层，当观察到事件完成时，相应的预填充请求立即合并到当前解码批次中。

3.3 竞争容忍估计器

当预填充和解码阶段进行空间复用时，可能会出现争用，尤其是来自非托管资源（例如内存带宽）的争用。我们首先分析空间复用下两个阶段之间的竞争，然后介绍我们的建模方法。

3.3.1 竞争分析。虽然 GreenContext 支持精确的计算资源分配，但它无法管理内存或网络带宽。特别是，缺乏有效的带宽管理技术。更糟糕的是，当前的 GPU 不公开带宽使用的运行时监控，并且预填充和解码阶段都会大量消耗带宽，从而使争用难以预测。

表 2. 预填充和解码阶段的计算分析。

	Attention	FFN
Prefill w/o cache	$O(Ld^2 + L^2d)$	$O(Ld^2)$
Prefill w/ cache	$O(nd^2 + Lnd)$	$O(nd^2)$
Decode	$O(d^2 + (r+1)d)$	$O(d^2)$

为了评估对执行速度减慢的影响，我们使用 Llama-8B 和 Llama-70B 广泛分析了多路复用下的预填充和解码。图 11 报告了具有 8 个 A100 和 8 个 H100 GPU 的服务器上的解码速度减慢。 x 轴表示分配给解码批次的 SM 数量，其余 SM 分配给预填充批次。对于每种配置，预填充批次的总上下文长度（重用 + new）范围为 1,024 到 128K 令牌，而解码批次的重用长度范围为 1,024 到 1,024K 令牌。此分析需要一周时间。

如图 11 所示，在不同的分区配置和 GPU 上，争用引起的速度下降范围从接近零到约 30%。硬件分区之间的巨大差异表明了争用减慢的固有的不可预测性。尽管由于架构相似，这两个模型在相同的 GPU 上表现出相似的减速趋势，但这一观察结果对争用建模没有帮助。同时，预填充的结果类似，但由于空间限制而被省略。

3.3.2 SLO 保证的最坏情况估计。为了降低在线服务中不可预测的争用导致 SLO 违规的风险，MuxWise 引入了专为 SLO 保证量身定制的最坏情况延迟估计方法。关键的观察结果是，精确的延迟预测并不是保证 SLO 的唯一方法。重要的是确保调度阶段的延迟（考虑到其分配的计算资源）不超过预定义的目标。因此，MuxWise 通过首先预测其单独运行的延迟，然后应用最大减速因子来执行最坏情况估计。

单人运行预测器。为了预测单独运行的延迟，我们分析了预填充和解码的计算复杂性，并使用离线分析构建了一个预测器。每次预填充或解码迭代的延迟由重用和新上下文的令牌长度确定。表 2 总结了批量大小为 1 时预填充和解码阶段的计算分析。关键因素如下:

- d：每个标记表示的隐藏维度。
- L：令牌总长度。
- r：重用（缓存）上下文的令牌长度。
- $n = L - r$：新上下文的标记长度。

基于表 2 中的复杂性分析，我们构建了预填充和解码阶段的延迟预测模型。预填充模型如公式 1 所示，解码模型如公式 2 所示，其中所有 θ 项均为系数。我们

separate the models because state-of-the-art serving frameworks adopt different execution paths for these two phases, including distinct GPU kernel implementations and launch methods.

$$T_{Prefill} = \theta_1 \cdot \sum_{i}^{bs} n_i^2 + \theta_2 \cdot \sum_{i}^{bs} n_i \cdot r_i + \theta_3 \cdot \sum_{i}^{bs} n_i + \theta_4 \quad (1)$$

$$T_{Decode} = \theta_1 \cdot \sum_{i}^{bs} r_i + \theta_2 \cdot bs + \theta_3 \quad (2)$$

The trained models achieve high accuracy, with a maximum deviation of 8.16% for prefill and 8.84% for decode, effectively supporting MuxWise's online scheduling. Meanwhile, the offline profiling for training the solo-run predictor can be completed within a few hours, which is acceptable. It is a one-time effort per LLM–machine pair and has been widely adopted in prior LLM serving work [1, 44, 53].

Contention guard. To provide the maximum slowdown factor, MuxWise introduces the contention guard. Specifically, the contention guard provides slowdown factors only for decode, which will be explained in §3.4.1. The contention guard is built using data collected through grid-sampling-based profiling. This profiling spans five variables: the number of new and reused tokens in prefill, the batch size, and total reused tokens in decode, and the partition configuration. For each pair of prefill and decode iterations to be estimated, the contention guard returns the maximum slowdown factor of the grid cell they fall into as the estimation result.

Building such a contention guard incurs much higher offline profiling overhead than the solo-run predictor, since pairwise profiling is needed to obtain slowdown data. Fortunately, extensive profiling shows that slowdown remains within a limited range, with a maximum of 20% on A100 GPUs and 30% on H100 GPUs. This indicates that even with coarse-grained profiling, worst-case latency inflation does not exceed 30%.

To this end, we initialize the contention guard using coarse-grained grid sampling. Specifically, we sample variables such as new and reused tokens in prefill, as well as single-request reused tokens in decode batches, at powers-of-4 granularity, ranging from 2K to 128K. The sampled decode batch sizes follow SOTA serving frameworks (around 20 batch sizes). We partition GPUs at the granularity of 16 SMs, yielding 6 configurations for A100 and 7 for H100. In total, the number of samples per LLM–machine pair is calculated as $(4\times4-1)\times4\times20\times6 \approx 7K$, which can be collected within 12 hours. In the equation, we exclude the case with 128K new and 128K reused tokens in prefill, since 128K is the maximum context window supported by mainstream LLMs [5, 15].

The reason for using 16 SMs as the granularity are twofold: (1) kernels on H100 and newer GPUs requires 16 SMs, due to using new features like thread block cluster [3], and (2) experiments indicate that 16 SMs already deliver strong performance improvements. Finer-grained scheduling offers little benefit while increasing memory overhead.

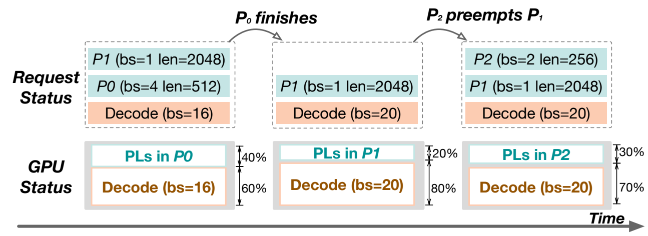


Figure 12. The dispatching policy of MuxWise.

Furthermore, MuxWise leverages runtime execution data to continuously update the contention guard, thus refining its SLO guarantees. Even with the coarse-grained contention guard, MuxWise already outperforms existing baselines in both SLO attainment and system goodput (§4).

3.4 SLO-aware Dispatcher

With bubble-less multiplexing and contention-tolerant modeling, we introduce MuxWise's detailed dispatching policy.

3.4.1 Priorities of prefill and decode. In this work, we focus on the scheduling within a single serving instance. In MuxWise, we prioritize SLO attainment for the decode phase and process the prefill phase as early as possible. SLO attainment for the prefill phase is not directly guaranteed for two reasons. Firstly, although we prioritize the decode phase, we only allocate just-enough compute resources for it. Since the remaining compute resources are allocated to the prefill phase, its SLO is generally expected to be met. Secondly, when SLO violations occur for the prefill phase, it indicates that the inference load has exceeded the peak capacity of the current LLM serving instance. In such cases, further scheduling efforts would no longer improve performance.

This is also why the contention guard in the contention-tolerant estimator only provides a maximum slowdown factor for decode. When predicting the prefill phase, MuxWise does not need an accurate or worst-case estimate. It only requires that the predicted latency of the launched prefill layers exceeds that of the corresponding decode iteration, ensuring full utilization of the allocated compute resources.

3.4.2 Dispatching policy. Building on the above analysis, Figure 12 illustrates MuxWise's SLO-aware dispatching policy. The system makes scheduling decisions after each prefill batch completes and at the end of each decode iteration. Specifically, the dispatcher selects prefill layers either from the ongoing prefill batch or from a new batch in the request queue, and allocates compute resources between the prefill and decode phases.

As shown in Figure 12, the dispatcher allocates a best-fit number of SMs (60%) to decode and assigns the remaining

将模型分开，因为最先进的服务框架在这两个阶段采用不同的执行路径，包括不同的 GPU 内核实现和启动方法。

$$T_{Prefill} = \theta_1 \cdot \sum_{i}^{bs} n_i^2 + \theta_2 \cdot \sum_{i}^{bs} n_i \cdot r_i + \theta_3 \cdot \sum_{i}^{bs} n_i + \theta_4 \tag{1}$$

$$T_{Decode} = \theta_1 \cdot \sum_{i}^{bs} r_i + \theta_2 \cdot bs + \theta_3 \tag{2}$$

训练后的模型准确率很高，预填充最大偏差为8.16%，解码最大偏差为8.84%，有效支持MuxWise的在线调度。同时，用于训练单独运行预测器的离线分析可以在几个小时内完成，这是可以接受的。这是每个 LLM-机器对的一次性工作，并且已在之前的 LLM 服务工作中广泛采用 [1, 44, 53]。

争用守卫。为了提供最大的减速因子，MuxWise 引入了争用防护。具体来说，争用防护仅为解码提供减速因子，这将在第 3.4.1 节中解释。争用防护是使用通过基于网格采样的分析收集的数据构建的。该分析涵盖五个变量：预填充中新的和重用的令牌的数量、批量大小、解码中重用的令牌总数以及分区配置。对于要估计的每对预填充和解码迭代，争用防护返回它们落入的网格单元的最大减速因子作为估计结果。

构建这样的争用防护会比单独运行的预测器产生更高的离线分析开销，因为需要成对分析来获取减速数据。幸运的是，广泛的分析表明，速度下降仍然在有限的范围内，A100 GPU 上最多下降 20%，H100 GPU 上最多下降 30%。这表明即使采用粗粒度分析，最坏情况下的延迟膨胀也不会超过 30%。

为此，我们使用粗粒度网格采样来初始化争用防护。具体来说，我们以 4 的幂粒度（范围从 2K 到 128K）对预填充中的新令牌和重用令牌以及解码批次中的单请求重用令牌等变量进行采样。采样的解码批量大小遵循 SOTA 服务框架（大约 20 个批量大小）。我们以 16 SM 的粒度对 GPU 进行分区，为 A100 提供 6 种配置，为 H100 提供 7 种配置。总的来说，每个LLM-机器对的样本数量计算为$(4 \times 4 - 1) \times 4 \times 20 \times 6 \approx 7K$，可以在12小时内收集。在等式中，我们排除了预填充中 128K 新令牌和 128K 重用令牌的情况，因为 128K 是主流 LLM 支持的最大上下文窗口 [5, 15]。

使用 16 个 SM 作为粒度的原因有两个：(1) H100 和更新的 GPU 上的内核需要 16 个 SM，因为使用了线程块集群 [3] 等新功能，以及 (2)

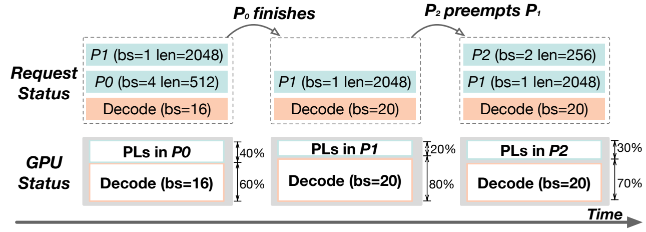


图 12.MuxWise 的调度策略。

实验表明 16 个 SM 已经实现了显着的性能改进。更细粒度的调度在增加内存开销的同时几乎没有什么好处。

此外，MuxWise 利用运行时执行数据来持续更新争用防护，从而完善其 SLO 保证。即使使用粗粒度的争用防护，MuxWise 在 SLO 实现和系统吞吐量方面也已经优于现有基线 (§4)。

3.4 SLO 感知调度程序

通过无气泡复用和容争建模，我们介绍了 MuxWise 的详细调度策略。

3.4.1 预填充和解码的优先级。在这项工作中，我们重点关注单个服务实例内的调度。在 MuxWise 中，我们优先考虑解码阶段的 SLO 实现，并尽早处理预填充阶段。由于两个原因，无法直接保证预填充阶段的 SLO 达到。首先，虽然我们优先考虑解码阶段，但我们只为其分配足够的计算资源。由于剩余的计算资源被分配给预填充阶段，因此通常期望能够满足其 SLO。其次，当预填充阶段发生 SLO 违规时，表明推理负载已超过当前 LLM 服务实例的峰值容量。在这种情况下，进一步的调度工作将不再提高性能。

这也是为什么竞争容忍估计器中的竞争保护只为解码提供最大减速因子的原因。在预测预填充阶段时，MuxWise 不需要准确或最坏情况的估计。它只要求启动的预填充层的预测延迟超过相应的解码迭代的延迟，从而确保分配的计算资源的充分利用。

3.4.2 调度策略。基于上述分析，图 12 说明了 MuxWise 的 SLO 感知调度策略。系统在每个预填充批次完成后以及每次解码迭代结束时做出调度决策。具体来说，调度程序从正在进行的预填充批次或请求队列中的新批次中选择预填充层，并在预填充和解码阶段之间分配计算资源。

如图 12 所示，调度程序分配最合适数量的 SM（60%）进行解码，并分配剩余的 SM

SMs (40%) to prefill phase $P0$. The resource partition satisfies the decode SLO and maximizes prefill throughput guided by the contention-tolerant estimator. To support layer-wise execution in the multiplex engine, the estimator computes the number of prefill layers to launch as $N_{PL} = \lceil (T_d \times N_T)/T_P \rceil$, where T_d is the estimated decode latency, T_P is the estimated prefill latency, and N_T is the number of transformer layers in the served LLM.

Once $P0$ finishes computation, it is merged into the decode batch, increasing the batch size to 20. The scheduler then retrieves a new prefill batch $P1$ and adjusts the partition to 20% SMs for prefill and 80% for decode to meet SLO targets.

Later, when a new prefill batch $P2$ arrives, it would normally wait for $P1$ to complete. However, because $P1$ has a long input length, this delay risks violating P2's SLO. To avoid this, MuxWise allows $P2$ to preempt $P1$, provided that preemption does not cause $P1$ to miss its own TTFT SLO.

MuxWise does not allow recursive preemption. For example, after $P2$ preempts prefill batch $P1$, no other batch may preempt $P2$. This design is reasonable, as short requests typically preempt long ones, and preempting a short request in turn would likely cause it to miss its SLO. MuxWise checks SLO attainment only when a prefill batch is preempted; otherwise, it prioritizes processing the active prefill batch as quickly as possible. Notably, preemption in MuxWise is optional. Even when disabled, MuxWise still delivers substantial performance improvements over the baselines.

4 Evaluation

4.1 Experimental Setup

Testbed. We mainly evaluate MuxWise on a server equipped with 8 A100-80GB GPUs. The GPUs are interconnected via NVLINK, providing 600 GB/s of bandwidth. We also evaluate MuxWise on two additional servers to demonstrate its effectiveness on newer GPUs and larger LLMs: one with 8 H100-SMX5-80GB GPUs and another with 8 H200-SMX5-141GB GPUs. These servers offer higher compute capability and larger GPU memory. All experiments are conducted with PyTorch 2.6.0 [31]. MuxWise is implemented using SGLang [52] version 0.4.10post2. The GPU driver version is 570.124.06, and the CUDA version is 12.8.

Models. We primarily evaluate MuxWise using two LLMs from the Llama family [15, 37, 38]: Llama-8B and Llama-70B. These models differ in size and represent the most commonly hosted LLMs in the cloud. We also evaluate a larger MoE model, Qwen3-235B with 22B activated, to demonstrate MuxWise's generality.

Baselines. We compare MuxWise against 3 state-of-the-art solutions for efficient LLM serving. Model parallelism techniques such as tensor parallelism [51] are employed to parallelize the deployed models. For MuxWise, we fix the tensor parallelism degree to 8. Details of each baseline's model parallelism configuration are provided when the baseline is introduced. For all systems, the KV cache memory pool is configured as large as possible to maximize throughput.

- **Chunked-prefill in SGLang [1]**: This version of SGLang is equipped with chunked-prefill, as proposed by SARATHI-Serve [1]. We follow SARATHI-Serve's methodology to calculate the token budget for each workload prior to experiments. It is offline tuned under specific TBT targets for each model. Unlike SARATHI-Serve, which serially executes prefill and decode attention kernels, SGLang leverages Flashinfer [46], a high-performance inference kernel library, to fuse them into a single kernel. It is expected to deliver performance similar to POD-attention [21].
- **NanoFlow [54]**: This is an enhanced version of chunked-prefill with operator-level intra-GPU multiplexing, targeting near-optimal throughput under a relatively loose SLO requirement (200 ms). It requires a large token budget (at least 1024) to achieve this goal. However, such a token budget cannot meet the SLO requirements of modern LLM serving ($\leq$ 100 ms). We use the same token budget as chunked-prefill for NanoFlow.
- **LoongServe [44]**: This is a dynamic disaggregated serving system. We adopt its model-parallelism configuration. For Llama-70B, sequence parallelism is set to 2 and tensor parallelism to 4. For Llama-8B, sequence parallelism is set to 4 and tensor parallelism to 2. It does not support new LLMs like MoE models.
- **Disaggregated serving in SGLang (SGLang-PD in short)**: This is the latest implementation of static disaggregation with KV-cache sharing across phases and requests. The P:D ratio is 1:1, with tensor parallelism set to 4 for each instance. DistServe [53] does not support KV-cache sharing across requests, making it unsuitable for modern LLM services. We also evaluated Dynamo [14], which performed substantially worse than SGLang-PD. Therefore, we use SGLang-PD as the state-of-the-art baseline of static disaggregation for evaluation.

Metrics. MuxWise targets goodput improvement. So, following prior works [1, 32, 34, 53], we use tail latency (e.g., P99) to assess SLO attainment. Meanwhile, there is also another metric to measure the SLO guarantee during decode phase: TPOT (timer per output token). In comparison, TBT accounts the latency of each individual token, whereas TPOT is an average metric that may mask the poor performance of some tokens [42]. Thus, we choose TBT over TPOT for a stricter SLO metric. We set the TBT SLO target to 50ms for Llama3-8B and 100ms for Llama3-70B, following prior works [1, 34]. We regard MuxWise's ability to deliver better TTFTs under skewed workloads as an additional benefit of the new serving paradigm. Moreover, it breaks the first-come-first-serve model used in other baselines. Thus, we only evaluate TTFT per token in §4.4.3.

SM (40%) 预填充阶段 $P0$。资源分区满足解码SLO并最大化由竞争容忍估计器引导的预填充吞吐量。为了支持多路复用引擎中的逐层执行，估计器计算要启动的预填充层的数量为$N_{PL} = \lceil (T_d \times N_T)/T_P \rceil$，其中$T_d$是估计的解码延迟，$T_P$是估计的预填充延迟，$N_T$是所服务的LLM中的转换器层的数量。

一旦 $P0$ 完成计算，它就会合并到解码批次中，将批次大小增加到 20。然后，调度程序检索新的预填充批次 $P1$ 并将分区调整为 20% SM 用于预填充，80% 用于解码，以满足 SLO 目标。

稍后，当新的预填充批次 $P2$ 到达时，它通常会等待 P 1 完成。然而，由于 $P1$ 的输入长度很长，这种延迟有违反 P2 的 SLO 的风险。为了避免这种情况，MuxWise 允许 $P2$ 抢占 $P1$，前提是抢占不会导致 $P1$ 错过其自己的 T TFT SLO。

MuxWise 不允许递归抢占。例如，在$P2$抢占预填充批次$P1$之后，没有其他批次可以抢占$P2$。这种设计是合理的，因为短请求通常会抢占长请求，而抢占短请求又可能会导致它错过其 SLO。仅当预填充批次被抢占时，M uxWise 才会检查 SLO 达到情况；否则，它会优先考虑尽快处理活动的预填充批次。值得注意的是，MuxWise 中的抢占是可选的。即使禁用时，MuxWise 仍可在基准基础上实现显着的性能改进。

4 评价

4.1 实验设置

试验台。我们主要在配备 8 个 A100-80GB GPU 的服务器上评估 MuxWise。 GPU 通过 NVLINK 互连，提供 600 GB/s 的带宽。我们还在另外两台服务器上评估了 MuxWis e，以展示其在较新的 GPU 和更大的 LLM 上的有效性：一台配有 8 个 H100-SMX5-80GB GPU，另一台配有 8 个 H200-SMX5-141GB GPU。这些服务器提供更高的计算能力和更大的 GPU 内存。所有实验均使用 PyTorch 2.6.0 [31] 进行。 MuxWise 是使用 SGLang [52] 版本 0.4.10post2 实现的。 GPU驱动版本为570.124.06，CUDA版本为12.8。

模型。我们主要使用来自 Llama 家族的两个大语言模型 [15,37,38] 来评估 MuxWise：Llama-8B 和 Llama-70B。这些模型大小不同，代表了云中最常见的托管大语言模型。我们还评估了一个更大的 MoE 模型，即激活 22B 的 Qwen3-235B，以证明 MuxWise 的通用性。

基线。我们将 MuxWise 与 3 种最先进的解决方案进行比较，以实现高效的 LLM 服务。采用张量并行[51]等模型并行技术来并行化部署的模型。对于 MuxWise，我们将张量并行度固定为 8。每个基线的模型并行度配置的详细信息在基线被提供时提供。介绍了。对于所有系统，KV缓存内存池配置得尽可能大，以最大化吞吐量。

- SGLang [1] 中的分块预填充：此版本的 SGLang 配备了分块预填充，如 SARATHI-Serve [1] 提出的那样。我们在实验前遵循 SARATHI-Serve 的方法来计算每个工作负载的代币预算。它是根据每个型号的特定 TBT 目标进行离线调整的。与连续执行预填充和解码注意力内核的 SARATHI-Serve 不同，SGLang 利用高性能推理内核库 Flashinfer [46] 将它们融合到单个内核中。预计它能提供与 POD-attention 类似的性能 [21]。
- NanoFlow [54]：这是分块预填充的增强版本，具有操作员级 GPU 内复用，目标是在相对宽松的 SLO 要求（200 毫秒）下接近最佳吞吐量。实现这一目标需要大量的代币预算（至少 1024）。然而，这样的代币预算无法满足现代 LLM 服务的 SLO 要求（≤ 100 毫秒）。我们使用与 NanoFlow 的分块预填充相同的代币预算。
- LoongServe [44]：这是一个动态分类服务系统。我们采用其模型并行配置。对于 Llama-70B，序列并行度设置为 2，张量并行度设置为 4。对于 Llama-8B，序列并行度设置为 4，张量并行度为 2。它不支持像 Mo E 模型这样的新 LLM。
- SGLang 中的分解服务（简称 SGLang-PD）：这是静态分解的最新实现，具有跨阶段和请求共享 KV 缓存。 P:D 比率为 1:1，每个实例的张量并行度设置为 4。 DistS erve [53] 不支持跨请求共享 KV 缓存，这使得它不适合现代 LLM 服务。我们还评估了 Dynamo [14]，它的性能比 SGLang-PD 差很多。因此，我们使用 SGLang-PD 作为静态分解的最先进的基线进行评估。

指标。 MuxWise 的目标是提高吞吐量。因此，根据先前的工作 [1,32,34,53]，我们使用尾部延迟（例如 P99 ）来评估 SLO 的实现情况。同时，还有另一个衡量解码阶段 SLO 保证的指标：TPOT（每个输出令牌的计时器 ）。相比之下，TBT 考虑了每个单独代币的延迟，而 T POT 是一个平均指标，可能掩盖某些代币的不良性能[4 2]。因此，为了更严格的 SLO 指标，我们选择 TBT 而不是 TPOT。遵循先前的工作 [1, 34]，我们将 Llama3-8 B 的 TBT SLO 目标设置为 50ms，将 Llama3-70B 设置为 100ms。我们认为 MuxWise 在倾斜的工作负载下提供更好的 TTFT 的能力是新服务范例的额外好处。此外，它打破了其他基线中使用的先到先服务模型。因此，我们仅评估第 4.4.3 节中每个代币的 TTFT。

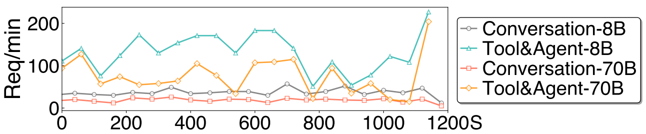


Figure 13. The two real-world workload traces after scaling.

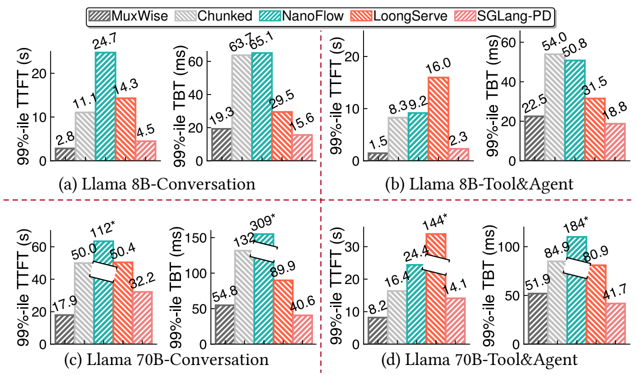


Figure 14. 99%-*ile* TTFT and TBT for Llama-8B and Llama-70B on real-world Conversation and Tool&Agent workloads. Chunked represents chunked-prefill in SGLang. Values marked with ∗ are too large; we clip their corresponding bars, so the bar height only decodes their relative size.

4.2 Evaluation on Real-world Workloads

4.2.1 End-to-end Performance. We begin by evaluating MuxWise with Llama-8B and Llama-70B under real-world workload traces. Figure 13 shows their request rates after scaling down, as they are originally from a large cluster. As illustrated, they show bursty request patterns (up to 13× spike within 1min).

Figure 14 shows the latency distribution of TTFT and TBT. Although the real-world traces are scaled down to a modest level, some baselines still easily reaches its peak throughput and enters an unstable state (NanoFlow in Figure 14-(c) and LoongServe in Figure 14-(d)). After omitting unstable results, MuxWise achieves average 99%-*ile* TTFT speedups of 3.57×, 5.98×, 4.65×, and 1.66× over chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively. MuxWise and the two disaggregated solutions consistently meet the TBT SLO, whereas chunked-prefill and NanoFlow fails in most cases. SGLang-PD achieves shorter TBT than MuxWise, as it statically reserves more compute resources for the decode instance.

Compared to chunked-prefill, MuxWise avoids the dilemma between SLO compliance and high utilization, bringing better performance for both prefill and decode phases. While tuning the token budget, we observe that either increasing or reducing it fails for SLO guarantee. This is because the reused length in prefill phase in the two workloads can reach up to 50K tokens. Further splitting the prefill into smaller chunks does not help control the TBT. MuxWise's PD multiplexing avoids this issue entirely.

Table 3. Results of other metrics for Llama-70B on Conversation workloads in Figure 14-(c).

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	TBT	Avg.	P50	Avg.	P50
MuxWise	**3.1**	**1.4**	**30.2**	**25.0**	**13.1**	**12.2**	**31.1**	**28.3**
Chunked	12.0	7.2	45.3	49.3	27.0	23.3	46.9	45.7
NanoFlow	51.4	52.3	105.6	98.9	83.4	84.2	120.2	103.2
LoongServe	17.7	14.7	61.0	58.3	38.4	35.8	62.3	60.6
SGLang-PD	7.38	3.95	32.9	32.5	18.4	16.4	33.5	33.2

NanoFlow performs worse than the original chunked-prefill. This is because, built atop chunked-prefill, NanoFlow is designed to overlap compute-bound kernels with memory-bound or communication kernels. To achieve this, it requires a large token budget (1024 in its paper) to ensure that chunked-prefill as a whole remains compute-bound. However, in Figure 14, the token budget has to be reduced to 256 to meet TBT SLO targets, where chunked-prefill is no longer compute-bound. The long reused context length in the two evaluated real-world traces further makes chunked-prefill harder to be compute-bound. Thus, NanoFlow degrades due to overlapping memory-bound kernels.

The situation worsen for NanoFLow with Llama-70B in Figure 14-(c&d). This could be attributed to the inherent model weight reload of intra-GPU overlapping. NanoFlow split each chunk into 2 nano batches , thus duplicating loading for each decode iteration [54]. When evaluating with Llama-70B, the reloading overhead is amplified due to the larger model size. Conversely, MuxWise duplicates loading only once during prefill, which typically co-runs with tens of decode iterations. Because prefill is compute-intensive and the reload is amortized over the entire phase, MuxWise imposes negligible bandwidth pressure, which is marginal relative to its overall benefits. While NanoFlow performs poorly on the two real-world workloads, it outperforms chunked-prefill for short input sequences without cross-request context length reuse(§4.3).

Against the two disaggregated solutions, MuxWise achieves significantly better TTFT. In LoongServe, instance scaling releases the KV cache needed for reuse in the prefill phase, causing redundant recomputation. In SGLang-PD, static disaggregation often leaves decode instances idle under fluctuating real-world workloads. In contrast, MuxWise avoids KV migration and adapts to dynamic workloads through intra-GPU compute partition reconfiguration.

4.2.2 Other Latency Metrics. In this experiment, we report results for other metrics, such as end-to-end latency and TPOT. We also present these results using other statistical measures, including average and P50 values. Table 3 shows the results on the Conversation workload with Llama-70B,

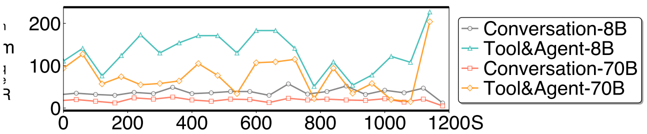
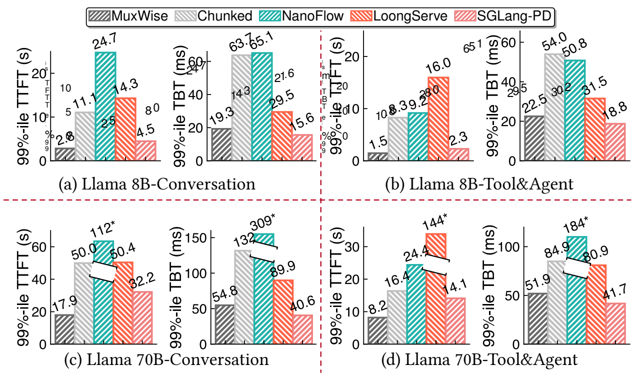


图 13. 扩展后的两个实际工作负载跟踪。



(a) Llama 8B-Conversation (b) Llama 8B-Tool&Agent

(c) Llama 70B-Conversation (d) Llama 70B-Tool&Agent

图 14. Llama-8B 和 Llama-70B 在实际对话和工具与代理工作负载上的 99%-ile TTFT 和 TBT。 Chunked 代表 SGLang 中的分块预填充。标有∗的值太大；我们剪裁它们相应的条形，因此条形高度仅解码它们的相对大小。

4.2 实际工作负载评估

4.2.1 端到端性能。我们首先在实际工作负载跟踪下使用 Llama-8B 和 Llama-70B 评估 MuxWise。图 13 显示了缩小规模后的请求率，因为它们最初来自大型集群。如图所示，它们显示了突发请求模式（1 分钟内最多 13 × 峰值）。

图 14 显示了 TTFT 和 TBT 的延迟分布。尽管现实世界的痕迹被缩小到适度的水平，但一些基线仍然很容易达到峰值吞吐量并进入不稳定状态（图14-（c）中的NanoFlow和图14-（d）中的LoongServe）。在忽略不稳定的结果后，MuxWise 比 chunked-prefill、NanoFlow、LoongServe 和 SGLang-PD 分别实现了 3.57×、5.98×、4.65 × 和 1.66× 的平均 99%-ile TTFT 加速。 MuxWise 和两种分类解决方案始终满足 TBT SLO，而分块预填充和 NanoFlow 在大多数情况下会失败。 SGLang-PD 实现的 TBT 比 MuxWise 更短，因为它为解码实例静态保留更多计算资源。

与分块预填充相比，MuxWise 避免了 SLO 合规性和高利用率之间的困境，为预填充和解码阶段带来了更好的性能。在调整代币预算时，我们发现增加或减少代币预算都无法保证 SLO。这是因为两个工作负载中预填充阶段的重用长度最多可达 50K 个令牌。进一步将预填充物分成更小的块

表 3. 图 14-(c) 中 Llama-70B 对话工作负载的其他指标结果。

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	TBT	Avg.	P50	Avg.	P50
MuxWise	**3.1**	**1.4**	**30.2**	**25.0**	**13.1**	**12.2**	**31.1**	**28.3**
Chunked	12.0	7.2	45.3	49.3	27.0	23.3	46.9	45.7
NanoFlow	51.4	52.3	105.6	98.9	83.4	84.2	120.2	103.2
LoongServe	17.7	14.7	61.0	58.3	38.4	35.8	62.3	60.6
SGLang-PD	7.38	3.95	32.9	32.5	18.4	16.4	33.5	33.2

无助于控制TBT。 MuxWise 的 PD 复用完全避免了这个问题。

NanoFlow 的性能比原始分块预填充差。这是因为，NanoFlow 构建在分块预填充之上，旨在将计算绑定内核与内存绑定或通信内核重叠。为了实现这一点，它需要大量的代币预算（论文中为 1024）以确保分块预填充作为一个整体仍然受计算限制。然而，在图 14 中，代币预算必须减少到 256 才能满足 TBT SLO 目标，其中分块预填充不再受计算限制。两个评估的现实世界跟踪中的长重用上下文长度进一步使得分块预填充更难以受计算限制。因此，NanoFlow 由于重叠的内存绑定内核而性能下降。

图 14-(c&d) 中使用 Llama-70B 的 NanoFlow 的情况更糟。这可能归因于 GPU 内重叠的固有模型权重重新加载。 NanoFlow 将每个块分成 2 个纳米批次，从而为每个解码迭代重复加载 [54]。使用 Llama-70B 进行评估时，由于模型尺寸较大，重新加载开销会放大。相反，MuxWise 在预填充期间仅重复加载一次，通常与数十次解码迭代共同运行。由于预填充是计算密集型的，并且重新加载在整个阶段中分摊，因此 MuxWise 带来的带宽压力可以忽略不计，相对于其整体优势而言，这是微不足道的。虽然 NanoFlow 在两个实际工作负载上表现不佳，但对于没有交叉请求上下文长度重用的短输入序列，它的性能优于分块预填充（第 4.3 节）。

与这两种分类解决方案相比，MuxWise 实现了明显更好的 TTFT。在LoongServe中，实例伸缩会释放预填充阶段重用所需的KV缓存，从而导致冗余的重新计算。在 SGLang-PD 中，静态分解通常会使解码实例在实际工作负载波动的情况下处于空闲状态。相比之下，MuxWise 避免了 KV 迁移，并通过 GPU 内计算分区重新配置来适应动态工作负载。

4.2.2 其他延迟指标。在这个实验中，我们报告了其他指标的结果，例如端到端延迟和 TPOT。我们还使用其他统计指标（包括平均值和 P50 值）来呈现这些结果。表 3 显示了 Llama-70B 的对话工作负载的结果，

Table 4. Results of other metrics for Llama-70B on Tool&Agent workloads in Figure 14-(d).

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	P50	Avg.	P50	Avg.	P50
MuxWise	**1.3**	**1.0**	**27.2**	24.1	**4.0**	**1.6**	**33.1**	**27.7**
Chunked	2.4	1.1	30.5	**21.2**	5.5	2.3	45.9	47.1
NanoFlow	2.8	1.2	58.8	42.4	7.4	2.5	70.3	62.6
LoongServe	59.9	56.0	52.4	50.8	65.2	61.0	56.2	54.6
SGLang-PD	2.1	1.5	31.6	31.0	5.2	2.3	37.1	33.7

while Table 4 shows the results on the Tool&Agent workload with Llama-70B in §4.2.1. Results in other settings are similar and are omitted due to space constraints.

As shown in the two tables, while MuxWise focuses on improving high goodput, it also consistently outperforms the baselines across the reported metrics. There is only one outlier in the P50 TBT of Table 4, and the values are very close. This can occur because the P50 TBT in chunked-prefill may correspond to the latency of a pure decode iteration.

4.2.3 SLO Attainment and Goodput. We also measure the SLO attainment of TBT and the corresponding goodput to evaluate the effectiveness of MuxWise in meeting SLO compliance. In this experiment, we extract requests from the `Tool&Agent` trace but replace their arrival timestamps with those generated by a Poisson process at varying rates, following prior work [44]. We stop testing once the serving system becomes unstable or fails to meet the TBT SLO target.

Figure 15 shows the SLO attainment results under gradually increasing workloads. Under the constraint of meeting the *99%-ile* SLO guarantees, MuxWise achieves 2.6×, 5.2×, 2.0×, and 1.3× higher goodput than chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively for Llama-8B; and 3.06×, 2.62×, and 1.62× higher than chunked-prefill, LoongServe, and SGLang-PD for Llama-70B. NanoFlow never meets the SLO even with a small chunk size of 64 for Llama-70B; therefore, the corresponding goodput improvement is omitted. Table 5 further shows the corresponding token thorughput and GPU utilization of MuxWise and baselines. GPU utilization is an aggregated metric reported by NVIDIA Nsight Systems, that reflects the fraction of active SMs as well as the utilization of intra-SM resources.

Chunked-prefill, and NanoFlow fails to meet the TBT SLO even at lower request rates than the other two baselines. This is because chunking is largely ineffective at reducing TBT in real-world LLM services, where cross-request interactions are common. Compared to LoongServe, MuxWise achieves higher goodput by avoiding recomputation in multi-turn requests. Compared to SGLang-PD, MuxWise achieves higher goodput through a larger KV-cache pool and reduced idleness caused by static disaggregation. Meanwhile, MuxWise achieves shorter TTFT across all cases (up to 9.16×).

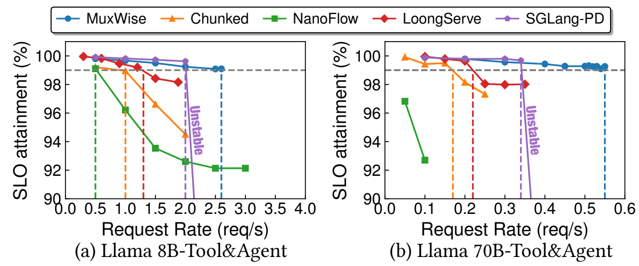


Figure 15. SLO attainment for Llama-8B and Llama-70B on Tool&Agent workload with varied request rates.

Table 5. Token throughput and GPU utilization for Llama-8B and Llama-70B on Tool&Agent workload under Goodput.

Model	Llama-8B		Llama-70B	
Metrics	Token/s	GPU Util.	Token/s	GPU Util.
MuxWise	25397	88.1	7430	84.0
Chunked	9768	63.8	2269	66.1
NanoFlow	4884	55.1	–	–
LoogServe	12698	75.3	2936	70.1
SGLang-PD	19535	P(72.4)/D(83.4)	4538	P(67.1)/D(81.9)

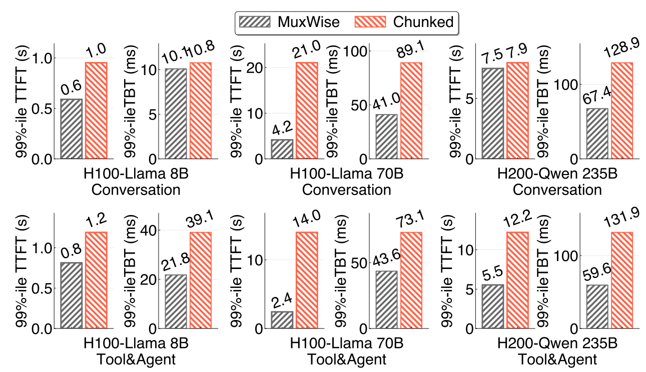


Figure 16. *99%-ile* TTFT and TBT for Llama-8B and Llama-70B on a server with 8 H100 GPUs and *99%-ile* TTFT and TBT for Qwen-235B on a server with 8 H200 GPUs.

4.2.4 More Advanced GPUs and Larger LLM. To demonstrate MuxWise's effectiveness on other GPUs and LLMs, we evaluate it with Llama-8B and Llama-70B on a server with 8 H100 GPUs, and with Qwen-235B on a server with H200 GPUs. In this experiment, we only compare MuxWise with chunked prefill. LoongServe does not support new MoE models like Qwen-235B, and disaggregated serving solutions are also infeasible for Qwen-235B, even though each H200 has 141 GB of GPU memory. Figure 16 shows the experimental results. Across all cases, MuxWise achieves an average 2.28× speedup on *99%-ile* TTFT and an average 1.81× speedup on *99%-ile* TBT. These consistent improvements demonstrate the generality of MuxWise's serving paradigm across diverse hardware and larger, newer LLMs.

表 4. Llama-70B 关于工具和代理工作负载的其他指标结果，如图 14-(d) 所示。

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	P50	Avg.	P50	Avg.	P50
MuxWise	**1.3**	**1.0**	**27.2**	24.1	**4.0**	**1.6**	**33.1**	**27.7**
Chunked	2.4	1.1	30.5	**21.2**	5.5	2.3	45.9	47.1
NanoFlow	2.8	1.2	58.8	42.4	7.4	2.5	70.3	62.6
LoongServe	59.9	56.0	52.4	50.8	65.2	61.0	56.2	54.6
SGLang-PD	2.1	1.5	31.6	31.0	5.2	2.3	37.1	33.7

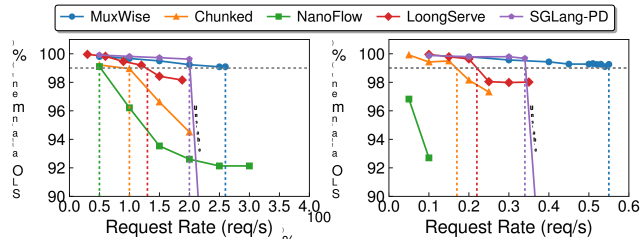
而表 4 显示了第 4.2.1 节中 Llama-70B 的工具和代理工作负载结果。其他设置的结果类似，由于空间限制而被省略。

如两个表所示，虽然 MuxWise 专注于提高高吞吐量，但它在报告的指标中也始终优于基线。表4的P50 TBT中只有1个异常值，并且值非常接近。发生这种情况是因为分块预填充中的 P50 TBT 可能对应于纯解码迭代的延迟。

4.2.3 SLO 达到和有效产出。我们还衡量 TBT 的 SLO 实现情况和相应的产出，以评估 MuxWise 在满足 SLO 合规性方面的有效性。在这个实验中，我们从工具和代理跟踪中提取请求，但按照先前的工作[44]，用泊松过程以不同速率生成的时间戳替换它们的到达时间戳。一旦服务系统变得不稳定或无法达到 TBT SLO 目标，我们就会停止测试。

图 15 显示了工作负载逐渐增加的情况下 SLO 达到的结果。在满足 99%-ile SLO 保证的约束下，MuxWise 的吞吐量分别比 Llama-8B 的 chunked-prefill、NanoFlow、LoongServe 和 SGLang-PD 高 2.6×、5.2×、2.0× 和 1.3×；Llama-70B 的性能比 chunked-prefill、LoongServe 和 SGLang-PD 高 3.06×、2.62× 和 1.62×。即使 Llama-70B 的块大小为 64，NanoFlow 也永远无法满足 SLO；因此，相应的吞吐量改进被忽略。表 5 进一步显示了 MuxWise 和基线的相应令牌吞吐量和 GPU 利用率。 GPU 利用率是 NVIDIA Nsight Systems 报告的聚合指标，反映了活动 SM 的比例以及 SM 内资源的利用率。

即使请求率低于其他两个基线，分块预填充和 NanoFlow 也无法满足 TBT SLO。这是因为在现实世界的 LLM 服务中，分块对于减少 TBT 来说基本上是无效的，在现实世界中，交叉请求交互很常见。与 LoongServe 相比，MuxWise 通过避免多轮请求中的重新计算来实现更高的吞吐量。与 SGLang-PD 相比，MuxWise 通过更大的 KV 缓存池实现了更高的吞吐量，并减少了静态分解引起的空闲。同时，MuxWise 在所有情况下都实现了更短的 TTFT（高达 9.16×）。



(a) Llama 8B-Tool&Agent (b) Llama 70B-Tool&Agent

图 15. Llama-8B 和 Llama-70B 在不同请求率的工具和代理工作负载上达到的 SLO。

表 5. Goodput 下工具和代理工作负载上 Llama-8B 和 Llama-70B 的令牌吞吐量和 GPU 利用率。

Model	Llama-8B		Llama-70B	
Metrics	Token/s	GPU Util.	Token/s	GPU Util.
MuxWise	25397	88.1	7430	84.0
Chunked	9768	63.8	2269	66.1
NanoFlow	4884	55.1	–	–
LoogServe	12698	75.3	2936	70.1
SGLang-PD	19535	P(72.4)/D(83.4)	4538	P(67.1)/D(81.9)

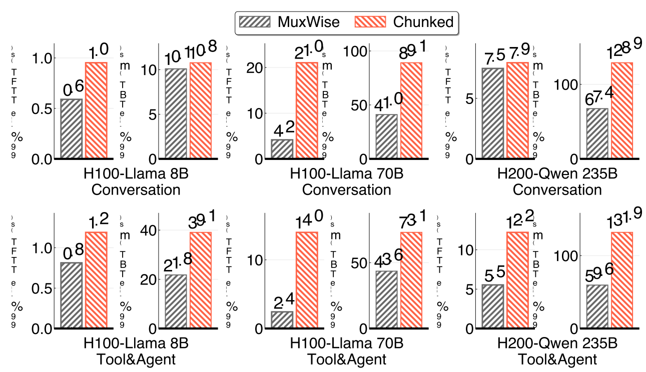


图 16. 具有 8 个 H100 GPU 的服务器上的 Llama-8B 和 Llama-70B 的 99%-ile TTFT 和 TBT，以及具有 8 个 H200 GPU 的服务器上的 Qwen-235B 的 99%-ile TTFT 和 TBT。

4.2.4 更先进的 GPU 和更大的大语言模型了证明 MuxWise 在其他 GPU 和 LLM 上的有效性，我们在具有 8 个 H100 GPU 的服务器上使用 Llama-8B 和 Llama-70B 以及在具有 H200 GPU 的服务器上使用 Qwen-235B 对其进行评估。在本实验中，我们仅将 MuxWise 与分块预填充进行比较。 LoongServe 不支持 Qwen-235B 等新的 MoE 型号，并且即使每个 H200 具有 141 GB GPU 内存，Qwen-235B 也无法实现分类服务解决方案。图 16 显示了实验结果。在所有情况下，MuxWise 在 99%-ile TTFT 上平均实现 2.28× 加速，在 99%-ile TBT 上平均实现 1.81× 加速。这些一致的改进证明了 MuxWise 服务范例在不同硬件和更大、更新的大语言模型通用性。

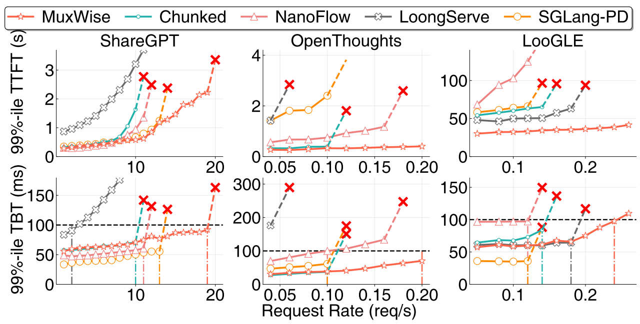


Figure 17. *99%-ile* TTFT and TBT with Llama-70B on three types of synthetic workloads.

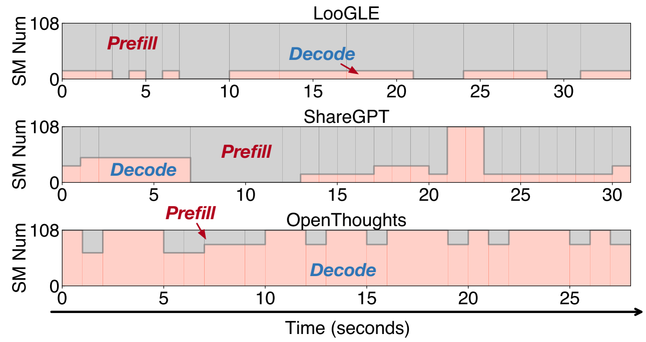


Figure 18. Change in compute partition between prefill and decode on LooGLE, ShareGPT, and OpenThoughts. Figures are sorted in descending order of prefill compute demand.

4.3 Evaluation on Diverse Synthetic Workloads

To better demonstrate MuxWise's effectiveness, we further evaluate it under three synthetic workloads. In the rest of the evaluation, we focus on Llama-70B due to space constraints, as results on other models are similar. Requests are generated by sampling inputs from ShareGPT [4], Openthoughts [17], and LooGLE [25], with arrival rates gradually increased following a Poisson process. Among these, only Openthoughts requests share a short system prompt. We select these workloads because they represent three typical patterns: moderate input and output, short input with ultra-long output, and ultra-long input with short output.

Figure 17 shows *99%-ile* TTFT and TBT of MuxWise and three baselines. On ShareGPT, MuxWise achieves goodput improvements of 1.9×, 1.73×, 9.5×, 1.46× over chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively. On LooGLE, it achieves 1.71×, 2×, 1.33×, 2× over the four baselines. On Open-Thoughts, it achieves the same 2× improvement over chunked-prefill, NanoFlow and SGLang-PD, while Loongserve never meets SLO.

On ShareGPT, MuxWise, chunked-prefill, NanoFlow, and SGLang-PD all provide SLO guarantees at the beginning. SGLang-PD even achieves better TBT than MuxWise, as it statically reserves more compute. In contrast, MuxWise delivers shorter TTFT by reserving only best-fit SMs for decode. On OpenThoughts, LoongServe performs worse than the others, as it is designed for long-context workloads rather than requests with short inputs and long outputs.

NanoFlow outperforms chunked-prefill only on ShareGPT. On OpenThoughts, the system spends most of the time in the decode phase. Therefore, NanoFlow splits decode iterations to enable overlapping, leading to higher TBT than chunked-prefill. On LooGLE, it performs worse due to the small token budget used for long requests.

We also observe that SGLang-PD performs much worse on OpenThoughts and LooGLE than on ShareGPT. The causes differ across workloads. For OpenThoughts, since requests share little context, the system must still reserve slots for KV caches during prefill and decode. As the request rate increases, prefill stalls once the KV cache pool runs out of space. For LooGLE, only four GPUs are available for prefill, causing requests to queue in the prefill instance.

4.3.1 Short Requests and Single GPU.

Running Llama-8B on an A100 with ShareGPT, MuxWise improves goodput by 1.2× over chunked-prefill while maintaining similar TBT. This is because even when chunking rarely happens, satisfying a strict TBT SLO still forces chunked-prefill to use a small token budget, limiting GPU utilization and peak goodput. Notably, real-world conversation inputs are becoming significantly longer (e.g., 1.2K and 2.3K tokens in two recent conversation traces from cloud vendors [35, 41], compared with 226 tokens in the older ShareGPT dataset). The conversation in our evaluation is a multi-turn real-world trace, whose average length approaches to 7.5K. This trend is driven by the widespread adoption of techniques such as RAG [24], which increase the effective model input length by appending retrieved sequences to the user input.

4.4 Ablation Study

4.4.1 Scheduling details of different tasks.

We further evaluate MuxWise's dynamic scheduling of compute partitions by extracting scheduling details from runtime serving in §4.3. As shown in Figure 18, MuxWise makes different scheduling decisions for different workloads. On LooGLE, most SMs are allocated to prefill, while on OpenThoughts, MuxWise allocates the majority of SMs to decode. Results on ShareGPT lie between LooGLE and OpenThoughts. Overall, however, more SMs are allocated to prefill on ShareGPT, since decode is typically memory-bound and does not require as many SMs as prefill. Notably, we use Figure 18 to show that different partitions are required for different workloads. They are relatively static because the request rate is stable. In real-world traces, the workload could be bursty. Experimental results show that, during a bursty interval in Figure 13, MuxWise activated all the six configurations within 30s.

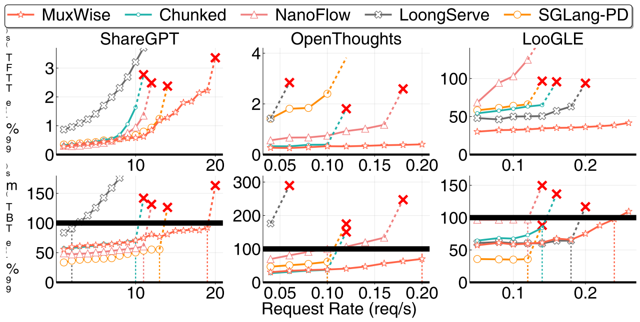


图 17. 使用 Llama-70B 在三种类型的合成工作负载上实现 99%-ile TTFT 和 TBT。

4.3 多种综合工作负载的评估

为了更好地证明 MuxWise 的有效性，我们在三种合成工作负载下进一步评估它。在其余的评估中，由于空间限制，我们重点关注 Llama-70B，因为其他模型的结果类似。请求是通过对来自 ShareGPT [4]、Openthoughts [17] 和 LooGLE [25] 的输入进行采样而生成的，到达率遵循泊松过程逐渐增加。其中，只有 Openthoughts 请求共享一个简短的系统提示。我们选择这些工作负载是因为它们代表了三种典型的模式：中等输入和输出、短输入超长输出、超长输入短输出。

图 17 显示了 MuxWise 的 99%-ile TTFT 和 TBT 以及三个基线。在 ShareGPT 上，MuxWise 的吞吐量分别比 chunked-prefill、NanoFlow、LoongServe 和 SGLang-PD 提高了 1.9×、1.73×、9.5×、1.46×。在 LooGLE 上，它在四个基线上实现了 1.71×、2×、1.33×、2×。在 OpenThoughts 上，它比 chunked-prefill、NanoFlow 和 SGLang-PD 实现了相同的 2× 改进，而 Loongserve 从未达到 SLO。

在 ShareGPT 上，MuxWise、chunked-prefill、NanoFlow 和 SGLang-PD 都在一开始就提供了 SLO 保证。 SGLang-PD 甚至比 MuxWise 实现了更好的 TBT，因为它静态地保留了更多计算量。相比之下，MuxWise 通过仅保留最适合的 SM 进行解码来提供更短的 TTFT。在 OpenThoughts 上，LoongServe 的表现比其他方案更差，因为它是为长上下文工作负载而不是短输入和长输出的请求而设计的。

NanoFlow 仅在 ShareGPT 上优于分块预填充。在 OpenThoughts 上，系统大部分时间都花在解码阶段。因此，NanoFlow 拆分解码迭代以实现重叠，从而导致比分块预填充更高的 TBT。在 LooGLE 上，由于用于长请求的代币预算较小，其性能较差。

我们还观察到 SGLang-PD 在 OpenThoughts 和 LooGLE 上的表现比在 ShareGPT 上差得多。不同工作负载的原因有所不同。对于 OpenThoughts，由于请求共享很少的上下文，系统仍然必须在预填充和解码期间为 KV 缓存保留插槽。作为请求率

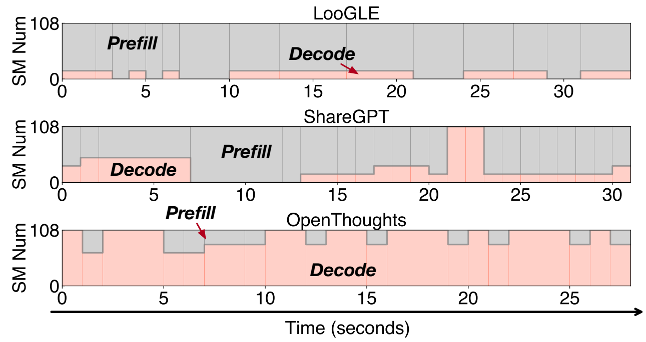


图 18. LooGLE、ShareGPT 和 OpenThoughts 上预填充和解码之间计算分区的变化。数字按预填充计算需求的降序排列。

一旦 KV 缓存池空间不足，预填充就会停止。对于 LooGLE，只有四个 GPU 可用于预填充，导致请求在预填充实例中排队。

4.3.1 短请求和单 GPU。 在具有 ShareGPT 的 A100 上运行 Llama-8B，MuxWise 的吞吐量比分块预填充提高了 1.2×，同时保持类似的 TBT。这是因为，即使分块很少发生，满足严格的 TBT SLO 仍然会迫使分块预填充使用较小的令牌预算，从而限制 GPU 利用率和峰值吞吐量。值得注意的是，现实世界的对话输入变得明显更长（例如，云供应商最近的两个对话跟踪中的 1.2K 和 2.3K 令牌 [35, 41]，而旧 ShareGPT 数据集中的 226 个令牌）。我们评估中的对话是多轮现实世界轨迹，其平均长度接近 7.5K。这一趋势是由 RAG [24] 等技术的广泛采用推动的，这些技术通过将检索到的序列附加到用户输入来增加有效模型输入长度。

4.4 消融研究

4.4.1 不同任务的调度细节。 我们通过从第 4.3 节中的运行时服务中提取调度详细信息，进一步评估 MuxWise 的计算分区动态调度。如图 18 所示，MuxWise 针对不同的工作负载做出不同的调度决策。在 LooGLE 上，大多数 SM 分配给预填充，而在 OpenThoughts 上，MuxWise 分配大部分 SM 来解码。 ShareGPT 的结果介于 LooGLE 和 OpenThoughts 之间。然而，总的来说，更多的 SM 被分配给 ShareGPT 上的预填充，因为解码通常受内存限制，并且不需要与预填充一样多的 SM。值得注意的是，我们使用图 18 来显示不同的工作负载需要不同的分区。它们相对静态，因为请求率稳定。在现实世界的跟踪中，工作负载可能是突发性的。实验结果表明，在图 13 中的突发间隔期间，MuxWise 在 30 秒内激活了所有六种配置。

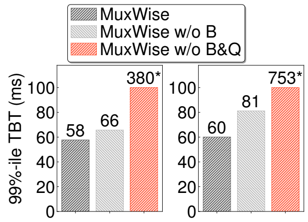


Figure 19. MuxWise with and without bubble-less multiplexing.

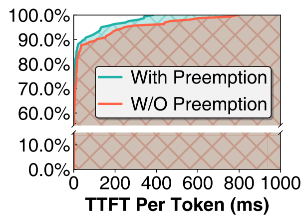


Figure 20. CDF of TTFT per token with and without preemption.

4.4.2 Effectiveness of Bubble-less Multiplex Engine.

As shown in Figure 9, bubbles commonly occur in the green context created for decode. In this experiment, we compare the TBT of MuxWise against its two variants. First, we disable layer-wise scheduling. Second, we further disable the query-based synchronization optimization. The workloads used are Tool&Agent under two different request rates.

Figure 19 presents the experimental results. As shown in the figure, disabling layer-wise execution slightly increases the TTFT of decode by approximately 10ms, which aligns with the typical kernel launch time for the prefill phase of Llama-70B. When query-based synchronization is further disabled, MuxWise suffers a significant degradation, 314ms for Llama-8B and 672ms for Llama-70B, due to frequent stalls waiting for the prefill phase to complete.

For further evaluation, we also collect the bubble ratio of MuxWise and chunked-prefill for the goodput results in Figure 15-a by profiling them with the NVIDIA Nsight Systems. The interval of the CUDA stream in the profiled timeline is treated as a bubble when it is not occupied by any GPU kernel. The bubble ratio is then defined as the proportion of all such bubbles in the compute stream. Since MuxWise has two active concurrent streams, we compute the bubble ratio for each stream and report their average as the final result. Notably, the bubble ratio is a temporal metric and does not reflect how GPU kernels utilize the parallel GPU resources they occupy.

MuxWise has a slightly higher bubble ratio (7.7% vs. 4.5%) due to its fine-grained kernel scheduling. These extra bubbles occurs when the system is purely processing decode iterations and all prefill layers are completed. Fortunately, these bubble do not degrade goodput, as there are no pending prefill launches and the decode iteration SLO is not violated. The reported GPU utilization in §4.2.3 also prove this.

4.4.3 Preemptive Scheduling for Long Request.

We evaluate the benefit of the bubble-less multiplex engine for preemptive scheduling by mixing requests from ShareGPT and LooGLE (50% each). Requests are generated at a rate of 0.5 per second following a Poisson process. Figure 20 shows the CDF of TTFT per token with and without preemptive scheduling. As shown, MuxWise achieves a 1.96× speedup on the 99%-*ile* TTFT per token, demonstrating that it can also be configured to support more advanced SLO-aware scheduling policies.

4.5 Overhead for Realizing PD-Multiplexing

Memory. MuxWise introduces some memory overhead by integrating GreenContext into existing serving systems. Creating a group of green contexts requires only 4MB, which is negligible compared to the total memory of modern GPUs. However, integrating it with CUDA Graph incurs a 6.2% overhead for both Llama-8B and Llama-70B on servers with 8 A100 or 8 H100 GPUs. This arises because the serving system records kernel launches for each decode-phase batch size into a CUDA Graph, consuming extra GPU memory. In MuxWise, there are six partition configurations in total, and each decode-phase compute partition created by GreenContext adds memory usage for all recorded batch sizes. Given the impressive performance gains of MuxWise, this overhead is acceptable.

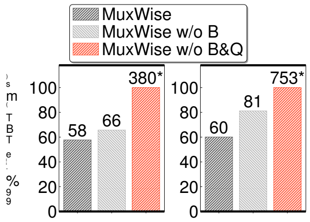
Runtime. MuxWise splits the prefill phase into multiple prefill layers to enable bubble-less scheduling. This may introduce extra overhead due to fine-grained kernel launches. We conduct an experiment to compare full prefill launching with layer-wise launching, where the prefill phase is split into the finest granularity. Across various configurations with different batch sizes and context lengths, the total overhead remains within 1.5%.

5 Discussion

Generality of MuxWise. MuxWise generalizes to accelerators that support intra-process spatial sharing with lightweight dynamic adjustment, such as GreenContext [10] on NVIDIA GPUs (supported since the Pascal architecture) and `hipExtStreamCreateWithCUMask()` on AMD GPUs.

Contexts where MuxWise excels. MuxWise targets scenarios with strict SLO guarantees (e.g., a decode-phase SLO below $100ms$). This is also the prevailing trend for achieving Model-as-a-Service in LLM serving. In this setting, MuxWise excels over existing works due to its efficient, fine-grained, and dynamic resource management between the prefill and decode phases. When the SLO target is loose or absent, such as in offline serving, MuxWise has no opportunity to outperform baselines such as chunked-prefill [55] or NanoFlow [54].

Large-scale deployment. While MuxWise is a single-instance optimization for high-goodput LLM serving, it can still benefit large-scale distributed deployments. In such deployments, MuxWise is complementary to disaggregated serving, as it optimizes each individual instance. Specifically, low-utilization decode instances could be replaced



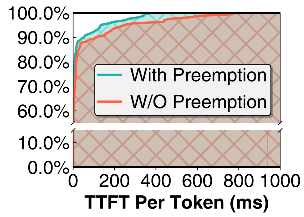


图 19. 使用和不使用无气泡复用的 MuxWise。图 20. 有和没有抢占情况下每个令牌的 TTFT 的 CDF。

4.4.2 无气泡多重发动机的有效性。如图 9 所示，气泡通常出现在为解码创建的绿色上下文中。在本实验中，我们将 MuxWise 的 TBT 与其两个变体进行比较。首先，我们禁用逐层调度。其次，我们进一步禁用基于查询的同步优化。使用的工作负载是两种不同请求率下的Tool &Agent。

图 19 显示了实验结果。如图所示，禁用逐层执行会使解码的 TTFT 略微增加约 10 毫秒，这与 Llama-70B 预填充阶段的典型内核启动时间一致。当进一步禁用基于查询的同步时，由于等待预填充阶段完成的频繁停顿，MuxWise 会出现显着的性能下降，Llama-8B 为 314ms，Llama-70B 为 672ms。

为了进一步评估，我们还通过使用 NVIDIA Nsight 系统对它们进行分析，收集了 MuxWise 和分块预填充的气泡比，以获得图 15-a 中的良好输出结果。当分析时间线中的 CUDA 流未被任何 GPU 内核占用时，该间隔被视为气泡。然后，气泡比率定义为计算流中所有此类气泡的比例。由于 MuxWise 有两个活动的并发流，因此我们计算每个流的气泡比，并将其平均值报告为最终结果。值得注意的是，气泡比是一个时间指标，并不反映 GPU 内核如何利用它们占用的并行 GPU 资源。

由于其细粒度的内核调度，MuxWise 的气泡率略高（7.7% vs. 4.5%）。当系统纯粹处理解码迭代并且所有预填充层完成时，就会出现这些额外的气泡。幸运的是，这些气泡不会降低吞吐量，因为没有待处理的预填充启动，并且没有违反解码迭代 SLO。 §4.2.3 中报告的 GPU 利用率也证明了这一点。

4.4.3 长请求的抢先调度。我们通过混合来自 ShareGPT 和 LooGLE 的请求（各 50%）来评估无气泡多路复用引擎对于抢占式调度的好处。请求按照泊松过程以每秒 0.5 个的速率生成。图 20 显示了使用和不使用抢占式调度时每个令牌的 TTFT 的 CDF。如图所示，MuxWise 实现了 1.96× 加速每个令牌的 99%-ile TTFT 上，表明它还可以配置为支持更高级的 SLO 感知调度策略。

4.5 实现 PD 复用的开销

记忆。 MuxWise 通过将 GreenContext 集成到现有服务系统中引入了一些内存开销。创建一组绿色上下文仅需要 4MB，与现代 GPU 的总内存相比，这可以忽略不计。然而，在具有 8 个 A100 或 8 个 H100 GPU 的服务器上，将其与 CUDA Graph 集成会导致 Llama-8B 和 Llama-70B 产生 6.2% 的开销。出现这种情况是因为服务系统将每个解码阶段批量大小的内核启动记录到 CUDA 图形中，从而消耗了额外的 GPU 内存。在 MuxWise 中，总共有六个分区配置，GreenContext 创建的每个解码阶段计算分区都会增加所有记录的批量大小的内存使用量。鉴于 MuxWise 令人印象深刻的性能提升，这种开销是可以接受的。

运行时。 MuxWise 将预填充阶段分为多个预填充层，以实现无气泡调度。由于细粒度的内核启动，这可能会带来额外的开销。我们进行了一项实验来比较完全预填充启动和分层启动，其中预填充阶段被分割成最细的粒度。在具有不同批量大小和上下文长度的各种配置中，总开销保持在 1.5% 以内。

5 讨论

MuxWise 的通用性。 MuxWise 泛化为通过轻量级动态调整支持进程内空间共享的加速器，例如 NVIDIA GPU 上的 GreenContext [10]（自 Pascal 架构开始支持）和 AMD GPU 上的 hipExtStreamCreateWithCUMask()。

MuxWise 擅长的环境。 MuxWise 针对具有严格 SLO 保证的场景（例如，解码阶段 SLO 低于 100ms）。这也是LLM服务中实现模型即服务的普遍趋势。在这种情况下，MuxWise 优于现有的工作，因为它在预填充和解码阶段之间进行高效、细粒度和动态的资源管理。当 SLO 目标松散或不存在时，例如在离线服务中，MuxWise 没有机会超越分块预填充 [55] 或 NanoFlow [54] 等基线。

大规模部署。虽然 MuxWise 是针对高吞吐量 LLM 服务的单实例优化，但它仍然可以使大规模分布式部署受益。在此类部署中，MuxWise 是对分类服务的补充，因为它优化了每个单独的实例。具体来说，可以替换低利用率的解码实例

with MuxWise instances to exploit idle resources via spatially multiplexing prefill. If prefill instance serves short requests—resulting in low utilization–or multiplexing decode on it does not violate TTFT SLO, it can be also utilized for higher efficiency. However, when prefill instances consistently handle long requests (e.g., using chunked pipeline parallelism [34]) or decode multiplexing violates TTFT SLO, MuxWise offers limited benefit.

6 Related Work

Multiplexing in LLM Serving. There are also prior works [16, 29] that multiplex the prefill and decode phases in LLM serving. WindServe [16] multiplexes prefill and decode using a normal CUDA stream, which leads to uncontrollable contention. It also does not address bubbles during scheduling. Our prototype implementation of WindServe shows that, on ShareGPT, MuxWise achieves a 1.61× goodput improvement under a 50*ms* TBT SLO on an A100 with Llama-8B. Tropical [29] replaces the decode instance in disaggregated serving with temporally multiplexed prefill and decode. It launches a full prefill only when sufficient slack exists. When developing MuxWise, we implemented an enhanced temporal-only variant that splits prefill into layers to fit small slacks. It performs at least 20% worse than MuxWise because it cannot spatially leverage wasted resources. There are also two similar community works [18]. Semi-PD [18] utilizes MPS for multiplexing. MPS enables *inter-process* spatial sharing but requires process restarts to adjust SM allocations. Semi-PD mitigates this by introducing a resident process and two additional inference engines, which adds significant complexity to existing frameworks. Bullet [28] relies on `libsmctrl` [7] to control SM allocation. While Bullet claims that it can dynamically change the SMs allocated to each CUDA graph, our trials with Bullet's open-sourced implementation show that the SM allocation for each CUDA graph does not change. This also aligns with the claim in `libsmctrl` [7] that it does not work with CUDA graph.

Compute management in LLM serving. There are two main approaches to compute management for improving system throughput under SLO constraints: disaggregation-based and fusion-based methods. On the one hand, DistServe [53] and Splitwise [32] disaggregate LLM serving into separate prefill and decode instances, while LoongServe [44] improves adaptability by enabling dynamic switching between the two at runtime. On the other hand, chunk-prefill [2] splits the prefill phase into chunks and fuses each chunk with a decode iteration for execution. However, disaggregation-based methods incur significant resource waste due to the coupled management of compute and memory, whereas fusion-based methods fail to fully maximize system throughput under SLO constraints. In contrast, our work decouples compute and memory management and maximizes goodput through spatial multiplexing.

Memory management in LLM serving. To improve system throughput in multi-turn or context-heavy LLM workloads, several systems propose memory management techniques. PagedAttention [23] introduces a paged memory pool to enable KV cache reuse between prefill and decode phases. Parrot [26] and SGLang [52] leverage context-aware caching to maximize reuse of KV segments across requests. MuxWise enhances these approaches by preserving memory sharing across phases and requests.

Execution time modeling. Performance modeling under spatial sharing is highly challenging, and prior efforts[22, 36, 48, 49] mainly focus on predicting interference for specific operators. GPUlet[9] uses linear regression with L1 cache utilization and DRAM bandwidth as input features to estimate performance interference among colocated operators. HSM[50] and GDP[20] also adopt linear regression based on low-level metrics in the simulator for operator slowdown prediction.

Compute partition techniques. Existing GPU partitioning techniques can be broadly categorized into time-sharing and space-sharing approaches. Time-sharing is typically implemented via API remoting [8, 27]. However, time-sharing alone is insufficient to meet MuxWise's requirements, as the prefill and decode phases already interleave in a time-sharing manner. In contrast, NVIDIA provides MPS [13], MIG [12], and GreenContext [10] for spatial sharing. MPS and MIG support inter-process spatial multiplexing, while GreenContext [10] enables intra-process spatial multiplexing with precise SM partition [11]. MuxWise builds on GreenContext to implement its PD multiplexing approach.

7 Conclusion

LLM services requires high goodput, yet existing serving systems struggle due to various deficiencies. To address these issues, we present MuxWise, an LLM serving framework with high goodput. MuxWise leverages a promising new serving paradigm, intra-GPU PD multiplexing, to achieve more flexible compute management for prefill and decode phases in LLM serving. Experiments show that MuxWise improves goodput by 2.2× on average over state-of-the-art baselines. Despite the notable performance improvement, MuxWise also introduces a simple yet effective design for current LLM serving systems. We plan to open-source MuxWise after publication.

Acknowledgments

This work is partially sponsored by the National Key Research and Development Program of China (2024YFB4505700), National Natural Science Foundation of China (62232011) and Natural Science Foundation of Shanghai Municipality (24ZR1430500). We thank the anonymous reviewers for their constructive feedback and suggestions.

与 MuxWise 实例一起通过空间复用预填充来利用空闲资源。如果预填充实例服务短请求（导致利用率低）或者其上的多路复用解码不违反 TTFT SLO，则也可以利用它来提高效率。然而，当预填充实例始终处理长请求（例如，使用分块管道并行性[34]）或解码复用违反 TTFT SLO 时，MuxWise 提供的好处有限。

6 相关工作

LLM 服务中的多路复用。还有一些先前的工作 [16, 29] 在 LLM 服务中复用了预填充和解码阶段。 WindServe [16] 使用普通 CUDA 流对预填充和解码进行多路复用，这会导致无法控制的争用。它也没有解决调度过程中的泡沫问题。我们的 WindServe 原型实现表明，在 ShareGPT 上，MuxWise 在使用 Llama-8B 的 A100 上以 50 *ms* TBT SLO 实现了 1.61× 良好投入改进。 Tropical [29] 用时间复用的预填充和解码替换了分散服务中的解码实例。仅当存在足够的松弛时，它才会启动完全预填充。在开发 MuxWise 时，我们实现了一种增强的仅时间变体，它将预填充分成几层以适应小的松弛。它的性能比 MuxWise 至少差 20%，因为它无法在空间上利用浪费的资源。还有两个类似的社区作品[18]。 Semi-PD [18] 利用 MPS 进行复用。 MPS 支持进程间空间共享，但需要进程重新启动来调整 SM 分配。 Semi-PD 通过引入驻留进程和两个额外的推理引擎来缓解这一问题，这大大增加了现有框架的复杂性。 Bullet [28] 依赖 libsmctrl [7] 来控制 SM 分配。虽然 Bullet 声称它可以动态更改分配给每个 CUDA 图的 SM，但我们对 Bullet 开源实现的试验表明，每个 CUDA 图的 SM 分配不会改变。这也与 libsmctrl [7] 中的声明一致，即它不适用于 CUDA 图。

LLM 服务中的计算管理。在 SLO 约束下提高系统吞吐量的计算管理方法主要有两种：基于分解的方法和基于融合的方法。一方面，DistServe [53]和Splitwise [32]将LLM服务分解为单独的预填充和解码实例，而LoongServe [44]通过在运行时启用两者之间的动态切换来提高适应性。另一方面，块预填充[2]将预填充阶段分割成块，并将每个块与解码迭代融合以供执行。然而，由于计算和内存的耦合管理，基于分解的方法会导致大量的资源浪费，而基于融合的方法无法在 SLO 约束下完全最大化系统吞吐量。相比之下，我们的工作将计算和内存管理解耦，并通过空间复用最大化吞吐量。

LLM 服务中的内存管理。为了提高多回合或上下文繁重的 LLM 工作负载中的系统吞吐量，一些系统提出了内存管理技术。 PagedAttention [23] 引入了分页内存池，以实现预填充和解码阶段之间的 KV 缓存重用。 Parrot [26] 和 SGLang [52] 利用上下文感知缓存来最大限度地跨请求重用 KV 段。 MuxWise 通过保留跨阶段和请求的内存共享来增强这些方法。

执行时间建模。空间共享下的性能建模非常具有挑战性，之前的工作[22,36,48,49]主要集中在预测特定运营商的干扰。 GPUlet[9]使用L1缓存利用率和DRAM带宽的线性回归作为输入特征来估计共置算子之间的性能干扰。 HSM[50]和GDP[20]还采用基于模拟器中低级指标的线性回归来进行操作员减速预测。

计算分区技术。现有的 GPU 分区技术可大致分为时间共享和空间共享方法。分时通常通过 API 远程处理来实现 [8, 27]。然而，仅分时还不足以满足 MuxWise 的要求，因为预填充和解码阶段已经以分时方式交错。相比之下，NVIDIA 提供 MPS [13]、MIG [12] 和 GreenContext [10] 用于空间共享。 MPS 和 MIG 支持进程间空间复用，而 GreenContext [10] 支持具有精确 SM 分区的进程内空间复用 [11]。 MuxWise 以 Green-Context 为基础来实现其 PD 复用方法。

7 结论

LLM 服务需要高产出，但现有的服务系统由于各种缺陷而陷入困境。为了解决这些问题，我们提出了 MuxWise，一个具有高吞吐量的 LLM 服务框架。 MuxWise 利用一种有前途的新服务范例，即 GPU 内 PD 复用，为 LLM 服务中的预填充和解码阶段实现更灵活的计算管理。实验表明，与最先进的基线相比，MuxWise 平均吞吐量提高了 2.2×。尽管性能显着提高，MuxWise 还为当前的 LLM 服务系统引入了简单而有效的设计。我们计划在发布后开源 MuxWise。

致谢

该工作得到国家重点研发计划（2024YFB4505700）、国家自然科学基金（62232011）和上海市自然科学基金（24ZR1430500）的部分资助。我们感谢匿名审稿人的建设性反馈和建议。

A Artifact Appendix

A.1 Abstract

MuxWise is an LLM serving framework adopting intra-GPU prefill-decode multiplexing, which is built on the top of SGLang[52]. We provide the source code of MuxWise and scripts to reproduce comparison of chunked-prefill. This appendix includes instructions for reproducing similar data in Figure 16 and Figure 17.

A.2 Artifact check-list (meta-information)

- **Model:** CodeLlama-34b-Instruct-hf.
- **Data set:** ShareGPT [4] and LooGLE [25].
- **Hardware:**
  NVIDIA H200 NVL (140 GB, 132 SMs)
  NVIDIA driver: 580.65.06 (must be greater than 570)
- **Experiments:** This appendix provides instrucitons for comparing 99%-*ile* TTFT and 99%-*ile* TBT MuxWise between MuxWise and chunked-prefill under various workload.
- **Metrics:** 99%-*ile* TTFT, 99%-*ile* TBT
- **Output:** Jsonl files containing metrics from MuxWise and chunked-prefill with different chunk size.
- **How much disk space required (approximately)?:** Approximately 200GB
- **How much time is needed to prepare workflow (approximately)?:** About 10 minutes to build from source code.
- **How much time is needed to complete experiments (approximately)?:** About 2 hours for ShareGPT workload and 4 hours for LooGLE workload.
- **Publicly available?:** Yes.

A.3 Description

A.3.1 How to access.

The source code of MuxWise is available for download on Zenodo: https://zenodo.org/records/18062118. The pre-built Docker image can be found in: https://hub.docker.com/layers/combathhhhhh/pdmux/sglpr_torch2.6_bench

A.3.2 Hardware dependencies.

Requires an x86-64 Linux host with at least 200 GB of free disk space, and an NVIDIA H200 NVL GPU (140 GB, 132 SMs).

A.3.3 Software dependencies.

NVIDIA driver: 580.65.06 (must be greater than 570).

A.3.4 Data sets.

ShareGPT: chatbot tasks, with an average input length of 226 and average output length of 195.

LooGLE: long-context understanding tasks, with an average input length of 30k and average output length of 15.

A.3.5 Models.

CodeLlama-34b-Instruct-hf.

A.4 Installation

Please follow the instructions below, which are adapted from our GitHub repository (https://github.com/ykcombat/sglang/tree/slo_config):

```
# 1. Clone the repository and switch to the slo_config branch
git clone https://github.com/ykcombat/sglang.git
cd sglang
git checkout slo_config

# 2. Build SGLang
pip install --upgrade pip
pip install -e "python"
```

A.5 Experiment workflow

Our experiments focus on comparison between MuxWise and chunked-prefill.

1. Download the required LLM model(CodeLlama-34b-Instruct-hf) to /workspace/data.
2. Start MuxWise or chunked-prefill server. You can change the environment virable $CHUNK_SIZE to start chunked-prefill server with different token budgets.

```
# 1. Start MuxWise Server
./start_pdmux.sh

# 2. Start Chunked-prefill Server
./start_chunk.sh
```

3. Start evaluating in another terminal.

```
# 1. Evaluate MuxWise on ShareGPT and LooGLE
./bench_pdmux.sh

# 2. Evaluate Chunked-prefill on ShareGPT and LooGLE
./bench_chunk.sh
```

A.6 Evaluation and expected results

When all experiments done, you will obtain jsonl files containing detailed metrics under /workspace/sglang. To visualize the results, run plot.ipynb; this will generate figures similar to the reference plot provided at /workspace/sglang/H200_result.png. Please note that results may vary depending on the specific hardware used. You can refer to https://github.com/ykcombat/sglang/blob/slo_config/README.md for more information.

A.7 Notes

When serving different workloads, different configurations are used, which can be found in our repository.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

神器附录

A.1 摘要

MuxWise 是一个采用 GPU 内预填充解码复用的 LLM 服务框架，它构建在 SGLang[52] 之上。我们提供了 MuxWise 的源代码和脚本来重现分块预填充的比较。本附录包括重现图 16 和图 17 中类似数据的说明。

A.2 工件检查表（元信息）

- 型号：CodeLlama-34b-Instruct-hf。
- 数据集：ShareGPT [4] 和 LooGLE [25]。
- 硬件：NVIDIA H200 NVL（140 GB，132 SM）
  NVIDIA 驱动程序：580.65.06（必须大于 570）
- 实验：本附录提供了在各种工作负载下比较 MuxWise 和分块预填充之间的 99%-ile TTFT 和 99%-ile TBT MuxWise 的说明。
- 指标：99%-ile TTFT、99%-ile TBT
- 输出：包含来自 MuxWise 的指标和具有不同块大小的分块预填充的 Jsonl 文件。
- 需要多少磁盘空间（大约）？： 大约 200GB
- 准备工作流程需要多少时间（大约）？： 从源代码构建大约需要 10 分钟。
- 完成实验需要多长时间（大约）？： ShareGPT 工作负载大约 2 小时，LooGLE 工作负载大约 4 小时。
- 公开吗？： 是的。

A.3 描述

A.3.1 如何访问。 MuxWise 的源代码可以在 Zenodo 上下载：https://zenodo.org/records/18062118。预构建的 Docker 镜像可以在：https://hub.docker.com/layers/combahhhhhh/pdmux/sglpr_torch2 中找到。 6_长凳

A.3.2 硬件依赖性。需要具有至少 200 GB 可用磁盘空间的 x86-64 Linux 主机和 NVIDIA H200 NVL GPU（140 GB，132 SM）。

A.3.3 软件依赖性。 NVIDIA 驱动程序：580.65.06（必须大于 570）。

A.3.4 数据集。 ShareGPT：聊天机器人任务，平均输入长度为 226，平均输出长度为 195。

LooGLE：长上下文理解任务，平均输入长度为30k，平均输出长度为15。

A.3.5 模型。 CodeLlama-34b-指令-hf。

A.4 安装

请按照以下说明进行操作，这些说明改编自我们的 GitHub 存储库 (https://github.com/ykcombat/sglang/tree/slo_config)：

```
# 1. 克隆存储库并切换到 slo_config 分支
git 克隆 https://github.com/ykcombat/sglang.git cd sglang git checkout slo_config

# 2. 构建 SGLang pip install --upgrade pip pip install -e "python"
```

A.5 实验流程

我们的实验重点是 MuxWise 和分块预填充之间的比较。

1. 下载所需的LLM模型（CodeLlama-34b- Instruct-hf）到/workspace/data。 2. 启动 MuxWise 或分块预填充服务器。您可以更改环境变量 $CHUNK_SIZE 以使用不同的令牌预算启动分块预填充服务器。

```
# 1.启动MuxWise服务器./start_pdmux.sh

# 2. 启动 Chunked-prefill 服务器 ./start_chunk.sh
```

3. 开始在另一个终端中评估。 # 1. 在 ShareGPT 和 LooGLE 上评估 MuxWise ./bench_pdmux.sh

```
# 2. 评估 ShareGPT 和 LooGLE 上的分块预填充 ./bench_chunk.sh
```

A.6 评价和预期结果

当所有实验完成后，您将在/workspace/sglang下获得包含详细指标的jsonl文件。要可视化结果，请运行plot.ipynb；这将生成类似于 /workspace/sglang/H200_result.png 中提供的参考图的图形。请注意，结果可能会因所使用的具体硬件而异。您可以参考https://github.com/ykcombat/sglang/blob/slo_config/README.md了解更多信息。

A.7 注释

当服务不同的工作负载时，会使用不同的配置，这些配置可以在我们的存储库中找到。

参考

[1] Amey Agrawal、Nitin Kedia、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav S. Gulavani、Alexey Tumanov 和 Ramachandran Ramjee。 2024. 使用 sarathi-serve 控制 LLM 推理中的吞吐量与延迟权衡。第 18 届 USENIX 操作系统设计与实现会议（美国加利福尼亚州圣克拉拉）会议记录 (OSDI'24)。美国 USENIX 协会，第 7 条，第 18 页。

[2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 (Aug. 2023). arXiv:2308.16369 [cs]

[3] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. 2025. NVIDIA Hopper Architecture In-Depth – Thread block clusters. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/#thread_block_clusters. Accessed 2026-01-10.

[4] anon8231489123. 2023. ShareGPT Vicuna Unfiltered – Cleaned Split (v3). https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_split.json. Accessed: 2025-04-16.

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, and et al. 2023. Qwen Technical Report. arXiv:2309.16609 (Sept. 2023). arXiv:2309.16609 [cs] doi:10.48550/arXiv.2309.16609

[6] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, et al. 2024. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204* (2024).

[7] Joshua Bakita and James H Anderson. 2023. Hardware compute partitioning on NVIDIA GPUs. In *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 54–66.

[8] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: QoS Awareness and Increased Utilization for Non-Preemptive Accelerators in Warehouse Scale Computers. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Atlanta Georgia USA, 681–696. doi:10.1145/2872362.2872368

[9] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 199–216.

[10] NVIDIA Corporation. 2025. CUDA Driver API: Green Contexts. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__GREEN__CONTEXTS.html. Accessed: 2025-03-29.

[11] NVIDIA Corporation. 2025. CUDA Runtime API: Stream Management. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__STREAM.html. Accessed: 2025-03-30.

[12] NVIDIA Corporation. 2025. Multi-Instance GPU (MIG). https://www.nvidia.com/en-sg/technologies/multi-instance-gpu/. Accessed: 2025-03-30.

[13] NVIDIA Corporation. 2025. *Multi-Process Service*. Version 570.

[14] NVIDIA Corporation. 2025. NVIDIA Dynamo: A Datacenter Scale Distributed Inference Serving Framework. https://github.com/ai-dynamo/dynamo. Accessed: 2025-04-07.

[15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, and et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 (Aug. 2024). arXiv:2407.21783

[16] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1283–1295. doi:10.1145/3695053.3730999

[17] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. 2025. OpenThoughts: data recipes for reasoning models. doi:10.48550/arXiv.2506.04178 arXiv:2506.04178 [cs].

[18] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. arXiv:2504.19867 [cs.CL] https://arxiv.org/abs/2504.19867

[19] Anysphere Inc. 2025. Cursor: The AI Code Editor. https://www.cursor.com/. Accessed: 2025-04-05.

[20] Magnus Jahre and Lieven Eeckhout. [n. d.]. Gdp: Using dataflow properties to accurately estimate interference-free performance at runtime. In *IEEE International Symposium on High Performance Computer Architecture (HPCA 2018)*. 296–309.

[21] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-attention: unlocking full prefill-decode overlap for faster LLM inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 897–912. doi:10.1145/3676641.3715996

[22] Sejin Kim and Yoonhee Kim. 2022. K-Scheduler: Dynamic Intra-SM Multitasking Management with Execution Profiles on GPUs. *Cluster Computing* 25, 1 (Feb. 2022), 597–617. doi:10.1007/s10586-021-03429-7

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, Koblenz Germany, 611–626. doi:10.1145/3600006.3613165

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[25] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2024. LooGLE: Can Long-Context Language Models Understand Long Contexts? arXiv:2311.04939 [cs.CL] https://arxiv.org/abs/2311.04939

[26] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.

[27] Yu-Shiang Lin, Chun-Yuan Lin, Che-Rung Lee, and Yeh-Ching Chung. 2019. qcuda: Gpgpu virtualization for high bandwidth efficiency. In *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 95–102.

[28] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2025. Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing. arXiv:2504.19516 [cs.DC] https://arxiv.org/abs/2504.19516

[29] Jinming Ma, Jiefei Chen, Xiuhong Li, Jiangfei Duan, Haojie Duanmu, Xingcheng Zhang, Chao Yang, and Dahua Lin. 2025. *Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing*. IEEE Press. https://doi.org/10.1109/DAC63849.2025.11132617

[30] OpenAI. 2025. ChatGPT. https://chatgpt.com/. Accessed: 2025-04-05.

[31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein,

[2] Amey Agrawal、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav S. Gulavani 和 Ramachandran Ramjee。 2023.SARATHI：通过分块预填充的搭载解码进行高效的 LLM 推理。 arXiv：2308.16369（2023 年 8 月）。 arXiv:2308.16369 [cs] [3] Michael Andersch、Greg Palmer、Ronny Krashinsky、Nick Stam、Vishal Mehta、Gonzalo Brito 和 Sridhar Ramaswamy。 2025. NVIDIA Hopper 架构深入研究 – 线程块集群。 https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-deep/#thread_block_clusters。访问时间：2026 年 1 月 10 日。 [4] anon8231489123。 2023.ShareGPT Vicuna 未过滤 – 清洁分割 (v3)。 https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_ split.json。访问时间：2025 年 4 月 16 日。 [5] 白金泽，白帅，褚云飞，崔泽宇，党凯，邓晓东，杨帆，葛文斌，韩宇，黄飞，惠斌元，罗吉，李梅，林俊阳，林润吉等。 2023.Qwen 技术报告。 arXiv：2309.16609（2023 年 9 月）。 arXiv:2309.16609 [cs] doi:10.48550/arXiv。 2309.16609 [6]白雨诗，涂尚清，张家杰，彭浩，王晓智，吕鑫，曹树林，徐家政，侯磊，董宇晓，等。 2024. LongBench v2：对现实的长上下文多任务进行更深入的理解和推理。 arXiv 预印本 arXiv:2412.15204 (2024)。 [7] 约书亚·巴基塔和詹姆斯·H·安德森。 2023 年。NVIDIA GPU 上的硬件计算分区。 2023 年 IEEE 第 29 届实时和嵌入式技术与应用研讨会 (RTAS)。 IEEE，54-66。 [8] 陈泉，杨海龙，杰森·马尔斯，唐凌嘉。 2016.Baymax：仓库规模计算机中非抢占式加速器的 QoS 意识和利用率提高。第二十一届编程语言和操作系统架构支持国际会议论文集。 ACM，美国佐治亚州亚特兰大，681–696。 doi:10.1145/2872362.2872368 [9] Seungbeom Choi、Sunho Lee、Yeonjae Kim、Jongse Park、Youngjin Kwon 和 Jaehyuk Huh。 2022 年。通过{时空}共享在{多 GPU} 服务器上提供异构机器学习模型。 2022 年 USENIX 年度技术会议（USENIX ATC 22）。 199–216。 [10] NVIDIA 公司。 2025. CUDA 驱动程序 API：绿色上下文。 https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA_ _GREEN__CONTEXTS.html。访问时间：2025 年 3 月 29 日。 [11] 英伟达公司。 2025.CUDA 运行时 API：流管理。 https://docs.nvidia.com/cuda/cuda-runtime-api/group_ _CUDART__STREAM.html。访问时间：2025 年 3 月 30 日。 [12] 英伟达公司。 2025。多实例 GPU (MIG)。 https://www. nvidia.com/en-sg/technologies/multi-instance-gpu/。访问时间：2025 年 3 月 30 日。 [13] 英伟达公司。 2025.多进程服务。版本 570。[14] NVIDIA 公司。 2025 年。NVIDIA Dynamo：数据中心规模分布式推理服务框架。 https://github.com/ai-dynamo/dynamo。访问时间：2025 年 4 月 7 日。 [15] Abhimanyu Dubey、Abhinav Jauhri、Abhinav Pandey、Abhishek Kadian、Ahmad Al-Dahle、Aiesha Letman、Akhil Mathur、Alan Schelten、Amy Yang、Angela Fan、Anirudh Goyal、Anthony Hartshorn、Aobo Yang、Archi Mitra、Archie Sravankumar 等。 2024. Llama 3 模特群。 arXiv：2407.21783（2024 年 8 月）。 arXiv:2407.21783 [16] Jingqi Feng、Yukai Huang、Rui Zhu、Si Cheng Liang、Ming Yan 和 Jie Wu。 2025. WindServe：采用基于流的动态调度的高效阶段分解大语言模型 第 52 届计算机体系结构国际研讨会 (ISCA '25) 会议记录。计算机协会，美国纽约州纽约市，1283–1295。 doi:10.1145/3695053.3730999 [17] Etash Guha、Ryan Marten、Sedrick Keh、Negin Raoof、Georgios Smyrnis、Hritik Bansal、Marianna Nezhurina、Jean Mercat、Trung Vu、Zayne Sprague、Ashima Suvarna、Benjamin Feuer、Liangyu Chen、Zaid Khan、Eric Frankel、Sachin格罗弗、Caroline Choi、Niklas Muennighoff、Shiye Su、赵万嘉、John Yang、Shreyas Pimpalgaonkar、Kartik Sharma、Charlie Cheng-Jie Ji、Yichuan Deng、Sarah Pratt、Vivek Ramanujan、Jon Saad-Falcon、Jeffrey Li、Achal Dave、Alon Albalak、Kushal Arora、Blake Wulfe、Chinmay Hegde、Greg Durrett、Sewoong Oh、Mohit Bansal、Saa-dia Gabriel、Aditya Grover、Kai-Wei Chang、Vaishaal Shankar、Aaron Gokaslan、Mike A. Merrill、Tatsunori Hashimoto、Yejin Choi、Jenia Jitsev、Reinhard Heckel、Maheswaran Sathiamoorthy、Alexandros G. Dimakis 和 Ludwig Schmidt。 2025. OpenThoughts：推理模型的数据配方。 doi:10.48550/arXiv.2506.04178 arXiv:2506.04178 [cs]。 [18] 洪柯，陈路芳，王忠，李秀红，毛秋丽，马建平，熊超，吴冠宇，韩不和，戴国豪，梁云，王宇。 2025.半PD：通过分阶段分解计算和统一存储实现高效的大语言模型 arXiv:2504.19867 [cs.CL] https://arxiv.org/abs/2504.19867 [19] Anysphere Inc. 2025。光标：AI 代码编辑器。 https://www.cursor。 com/.访问时间：2025 年 4 月 5 日。 [20] 马格努斯·贾尔和利文·埃克豪特。 [n. d.]。 Gdp：使用数据流属性来准确估计运行时的无干扰性能。在 IEEE 国际高性能计算机架构研讨会 (HPCA 2018) 中。 296–309。 [21] Aditya K. Kamath、Ramya Prabhu、Jayashree Mohan、Simon Peter、Ramachandran Ramjee 和 Ashish Panwar。 2025. POD-attention：解锁完整的预填充解码重叠，以实现更快的 LLM 推理。第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集，第 2 卷 (ASPLOS '25)。计算机协会，美国纽约州纽约市，897-912。 doi:10.1145/3676641.3715996 [22] Sejin Kim 和 Yoonhee Kim。 2022. K-Scheduler：在 GPU 上使用执行配置文件进行动态 SM 内多任务管理。集群计算 25, 1（2022 年 2 月）, 597–617。 doi:10.1007/s10586-021-03429-7 [23] Woosuk Kwon、Zhuohan Li、Siyuan Zhuang、Ying Shen、Lianmin Cheng、Codyhao Yu、Joseph Gonzalez、Hao Zhu 和 Ion Stoica。 2023.使用 PagedAttention 进行大型语言模型的高效内存管理。第 29 届操作系统原理研讨会论文集。 ACM，德国科布伦茨，611–626。 doi:10.1145/3600006.3613165 [24] Patrick Lewis、Ethan Perez、Aleksandra Piktus、Fabio Petroni、Vladimir Karpukhin、Naman Goyal、Heinrich Küttler、Mike Lewis、Wen-tau Yih、Tim Rocktäschel 等。 2020.知识密集型 NLP 任务的检索增强生成。神经信息处理系统的进展 33 (2020), 9459–9474。 [25] 李嘉琪，王萌萌，郑子龙，张慕涵。 2024.LooGLE：长上下文语言模型能否理解长上下文？ arXiv:2311.04939 [cs.CL] https://arxiv.org/abs/2311.04939 [26] 林超凡、韩振华、张成瑞东、杨雨清、杨范、陈晨和邱莉莉。 2024.Parrot：利用语义变量对基于 LLM 的应用程序进行高效服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。 929–945。 [27] 林玉祥、林春元、李志荣、钟业清。 2019.qcuda：Gpgpu 虚拟化，实现高带宽效率。 2019年IEEE云计算技术与科学国际会议（CloudCom）。 IEEE，95–102。 [28] 林泽佳，徐宏鑫，陈冠一，陈志光，陆雨桐，张贤伟。 2025。通过时空 GPU 资源共享促进 LLM 服务。 arXiv：2504.19516 [cs.DC] https：//arxiv.org/abs/2504.19516 [29]马金明，陈杰飞，李秀红，段江飞，端木豪杰，张兴成，杨朝和林大华。 2025. Tropical：通过 SLO 感知多路复用提高分类大语言模型中的 SLO 成就。 IEEE 出版社。 https://doi.org/10.1109/DAC63849.2025.11132617 [30] OpenAI。 2025.ChatGPT。 https://chatgpt.com/。访问时间：2025 年 4 月 5 日。 [31] Adam Paszke、Sam Gross、Francisco Massa、Adam Lerer、James Bradbury、Gregory Chanan、Trevor Killeen、Zeming Lin、Natalia Gimelshein、

Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, and et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 8026–8037.

[32] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019

[33] Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. 2024. Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solvers. arXiv:2408.06195 (Aug. 2024). arXiv:2408.06195

[34] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — a KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.

[35] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: designing LLM inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1348–1362. doi:10.1109/HPCA61900.2025.00102 ISSN: 2378-203X.

[36] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-Aware, Fine-Grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 1075–1092. doi:10.1145/3627703.3629578

[37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 (Feb. 2023). arXiv:2302.13971 doi:10.48550/arXiv.2302.13971

[38] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. In *Advances in Neural Information Processing Systems*. NeurIPS, Long Beach, CA, USA. arXiv:1706.03762 doi:10.48550/arXiv.1706.03762

[40] vLLM Team. [n. d.]. CUDA Graphs — vLLM Design Documentation. https://docs.vllm.ai/en/stable/design/cuda_graphs/. Accessed: 2026-01-09.

[41] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache cache in the wild: characterizing and optimizing KVCache cache at a large cloud provider. 465–482. https://www.usenix.org/conference/atc25/presentation/wang-jiahao

[42] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Zhonghui Zhang, Nguyen Cam-Tu, Rong Gu, Chen Tian, Guihai Chen, and Sheng Zhong. 2025. Revisiting Service Level Objectives and System Level Metrics in Large Language Model Serving. arXiv:2410.14257 [cs.LG] https://arxiv.org/abs/2410.14257

[43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[44] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 640–654. doi:10.1145/3694715.3695948

[45] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098

[46] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Eighth Conference on Machine Learning and Systems*.

[47] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

[48] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *Proceedings of the Twentieth European Conference on Computer Systems (ACM Conferences)*. 573–588. doi:10.1145/3689031.3696070

[49] Wei Zhang, Weihao Cui, Kaihua Fu, Quan Chen, Daniel Edward Mawhirter, Bo Wu, Chao Li, and Minyi Guo. 2019. Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters. In *Proceedings of the ACM International Conference on Supercomputing (ICS '19)*. Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3330345.3330351

[50] Xia Zhao, Magnus Jahre, and Lieven Eeckhout. [n. d.]. HSM: A Hybrid Slowdown Model for Multitasking GPUs. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2022)*. 1371–1385.

[51] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[52] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2000, 27 pages.

[53] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model

Luca Antiga、Alban Desmaison、Andreas Köpf、Edward Yang、Zach DeVito、Martin Raison 等。 2019.PyTorch：命令式高性能深度学习库。第 33 届国际神经信息处理系统会议论文集。 Curran Associates Inc.，美国纽约州雷德胡克，8026–8037。 [32] Pratyush Patel、Esha Choukse、Chaojie 张、Aashaka Shah、Íñigo Goiri、Saeed Maleki 和 Ricardo Bianchini。 2024.Splitwise：使用相分裂的高效生成大语言模型 2024 年 ACM/IEEE 第 51 届计算机体系结构国际研讨会 (ISCA)。 118-132。 doi:10.1109/ISCA59077.2024.00019 [33] 齐振廷，马明远，徐嘉航，张莉娜，杨范，杨毛。 2024 年。相互推理使规模较小的法学硕士成为更强大的问题解决者。 arXiv：2408.06195（2024 年 8 月）。 arXiv:2408.06195 [34] 秦若雨、李哲明、何蔚然、崔家磊、任峰、张明星、吴永伟、郑伟民、徐欣然。 2025. Mooncake：用更多的存储换取更少的计算——一种以 KVCache 为中心的架构，用于服务 LLM 聊天机器人。第 23 届 USENIX 文件和存储技术会议 (FAST 25)。 155–170。 [35] Jovan Stojkovic、张超杰、Íñigo Goiri、Josep Torrelas 和 Esha Choukse。 2025. DynamoLLM：设计 LLM 推理集群以提高性能和能源效率。 2025 年 IEEE 国际高性能计算机架构研讨会 (HPCA)。 1348–1362。 doi：10.1109/HPCA61900.2025.00102 ISSN：2378-203X。 [36] Foteini Strati，马贤哲，Ana Klimovic。 2024. Orion：用于 ML 应用程序的干扰感知、细粒度 GPU 共享。第十九届欧洲计算机系统会议 (EuroSys '24) 的会议记录。计算机协会，美国纽约州纽约市，1075–1092。 doi:10.1145/3627703.3629578 [37] Hugo Touvron、Thibaut Lavril、Gautier Izacard、Xavier Martinet、Marie-Anne Lachaux、Timothée Lacroix、Baptiste Rozière、Naman Goyal、Eric Hambro、Faisal Azhar、Aurelien Rodriguez、Armand Joulin、Edouard格雷夫和纪尧姆·兰普尔。 2023.LLaMA：开放高效的基础语言模型。 arXiv：2302.13971（2023 年 2 月）。 arXiv:2302.13971 doi:10.48550/arXiv.2302.13971 [38] Hugo Touvron、Louis Martin、Kevin Stone、Peter Albert、Amjad Almahairi、Yasmine Babaei、Nikolay Bashlykov、Soumya Batra、Prajjwal Bhargava、Shruti Bhosale、Dan Bikel、Lukas Blecher、Cristian Canton Ferrer、Moya Chen、Guillem Cucurull、David Esiobu、Jude Fernandes、Jeremy Fu、Wenyin Fu、Brian Fuller、Cynthia Gau、Vedanuj Goswami、Naman Goyal、Anthony Hartshorn、Saghar Hosseini、Rui Hou、Hakan Inan、Marcin Kardas、Viktor Kerkez、Madian Khabsa、Isabel Kloumann、Artem Korenev、Punit Singh Koura、Marie-Anne Lachaux、Thibaut Lavril、Jenya Lee、Diana Liskovich、陆英海、Yuning Mao、Xavier Martinet、Todor Mihaylov、Pushkar Mishra、Igor Molybog、Yixin Nie、Andrew Poulton、Jeremy Reizenstein、Rashi Rungta、Kalyan Saladi、Alan Schelten、 Ruan Silva、Eric Michael Smith、Ranjan Subramanian、Xiaoqing Ellen Tan、Binh Tang、Ross Taylor、Adina Williams、Jian Xiang Kuan、徐朴新、郑彦、Iliyan Zarov、张雨辰、Angela Fan、Melanie Kambadur、Sharan Narang、Aurelien Rodriguez、Robert Stojnic、Sergey Edunov 和 Thomas Scialom。 2023. Llama 2：开放基础和微调聊天模型。 arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288 [39] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N. Gomez、Lukasz Kaiser 和 Illia Polosukhin。 2023. 你所需要的就是注意力。神经信息处理系统的进展。 NeurIPS，美国加利福尼亚州长滩。 arXiv：1706.03762 doi：10。 48550/arXiv.1706.03762 [40] vLLM 团队。 [n. d.]。 CUDA 图形 — vLLM 设计文档。 https://docs.vllm.ai/en/stable/design/cuda_graphs/。访问时间：2026 年 1 月 9 日。 [41] 王嘉豪，韩金波，魏兴达，沉思杰，张定彦，方晨光，陈榕，于文渊，陈海波。 2025. 野外 KVCache 缓存：在大型云提供商处表征和优化 KVCache 缓存。 465–482。 https://www.usenix.org/会议/atc25/演讲/wang-jiahao

[42] 王志斌，李世鹏，周宇航，李雪，张中辉，Nguyen Cam-Tu，顾荣，田辰，陈桂海，钟胜。 2025。重新审视大型语言模型服务中的服务级别目标和系统级别指标。 arXiv:2410.14257 [cs.LG] https: //arxiv.org/abs/2410.14257 [43] Jason Wei、Xuezhi Wang、Dale Schuurmans、Maarten Bosma、Brian Ichter、Fei Xia、Ed H Chi、Quoc V Le 和 Denny Zhou。 2022.思想链提示引发大型语言模型中的推理。神经信息处理系统进展 35 (2022), 24824–24837. [44] 吴冰阳，刘胜宇，钟银民，孙鹏，刘轩哲，金鑫。 2024. LoongServe：通过弹性序列并行高效服务长上下文大型语言模型。在 ACM SIGOPS 第 30 届操作系统原理研讨会 (SOSP '24) 的会议记录中。计算机协会，美国纽约州纽约市，640–654。 doi:10.1145/3694715.3695948 [45] Jiayi Yao，Hanchen Li，Yuhan Liu，Siddhant Ray，Yihua Cheng，Qizheng Zhang，Kuntai Du，Shan Lu 和 Junchen Jiang。 2025.CacheBlend：通过缓存知识融合为 RAG 提供快速大型语言模型。第二十届欧洲计算机系统会议（荷兰鹿特丹）会议记录（EuroSys '25）。计算机协会，美国纽约州纽约市，94-109。 doi:10.1145/3689031.3696098 [46] Zihao Ye、Lequn Chen、Ruihang Lai、Wuwei Lin、Yineng Zhang、Stephanie Wang、Tianqi Chen、Baris Kasikci、Vinod Grover、Arvind Krishnamurthy 和 Luis Ceze。 2025.FlashInfer：用于 LLM 推理服务的高效且可定制的注意力引擎。第八届机器学习和系统会议。 [47] Gyeong-In Yu、Joo Seong Jeong、Geon-Woo Kim、Soojeong Kim 和 Byung-Gon Chun。 2022.Orca：基于 Transformer 的生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)。 521–538。 [48] 张树来，陈全，崔伟豪，赵涵，薛春雨，郑振，林伟，郭敏仪。 2025。通过自适应无泡时空共享提高 GPU 共享性能。在第二十届欧洲计算机系统会议（ACM 会议）的会议记录中。 573–588。 doi:10.1145/3689031.3696070 [49] 张伟，崔伟豪，付凯华，陈泉，Daniel Edward Mawhirter，吴波，李超，郭敏仪。 2019 年。Laius：提高数据中心的延迟意识并提高空间多任务加速器的利用率。在 ACM 国际超级计算会议 (ICS '19) 的会议记录中。计算机协会，美国纽约州纽约市，58-68。 doi:10.1145/3330345.3330351 [50] 夏昭，Magnus Jahre，和 Lieven Eeckhout。 [n. d.]。 HSM：多任务 GPU 的混合减速模型。第二十五届编程语言和操作系统架构支持国际会议 (ASPLOS 2022) 论文集。 1371–1385。 [51] 郑连民，李卓瀚，张浩，庄永浩，陈志峰，黄艳萍，王一达，徐远中，卓丹阳，Eric P. Xing，Joseph E. Gonzalez，Ion Stoica。 2022. Alpa：分布式深度学习的算子间和算子内并行性的自动化。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)。 559–578。 [52] 郑连民、尹良胜、谢志强、孙初跃、黄杰夫、于浩、曹诗一、Christos Kozyrakis、Ion Stoica、Joseph E. Gonzalez、Clark Barrett、盛英。 2024.SGLang：结构化语言模型程序的高效执行。第 38 届神经信息处理系统国际会议（加拿大不列颠哥伦比亚省温哥华）（NIPS '24）论文集。 Curran Associates Inc.，美国纽约州雷德胡克，文章 2000 年，27 页。 [53] 钟银民，刘胜宇，陈俊达，胡建波，朱一波，刘玄哲，金鑫，张浩。 2024.DistServe：针对 Goodput 优化的大型语言模型分解预填充和解码

Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

[54] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. 749–765. https://www.usenix.org/conference/osdi25/presentation/zhu-kan

[55] Simiao Zuo, Xiaodong Liu, Jian Jiao, Young Jin Kim, Hany Hassan, Ruofei Zhang, Tuo Zhao, and Jianfeng Gao. 2022. Taming Sparsely Activated Transformer with Stochastic Experts. arXiv:2110.04260 (Feb. 2022). arXiv:2110.04260 [cs]

服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。193–210。 [54] 朱侃，高宇飞，赵一龙，赵良宇，左格飞，顾一乐，谢德东，叶子豪，镰堀圭介，林建宇，王子仁，斯蒂芬妮·王，Arvind Krishnamurthy，Baris Kasikci。 2025.NanoFlow：迈向服务吞吐量的最佳大型语言模型。 749–765。 https://www.usenix.org/conference/osdi25/presentation/zhu-kan

[55]左思苗，刘晓东，焦健，Young Jin Kim，Hany Hassan，张若飞，赵拓，高剑锋。 2022. 与随机专家一起驯服稀疏激活的变压器。 arXiv ：2110.04260（2022 年 2 月）。 arXiv:2110.04260 [cs]